

ГЛАВА 6

ФУНКЦИИ

В этой главе...

6.1. Основы функций	267
6.2. Передача аргументов	275
6.3. Типы возвращаемого значения и оператор <code>return</code>	292
6.4. Перегруженные функции	302
6.5. Специальные средства	308
6.6. Подбор функции	317
6.7. Указатели на функции	322
Резюме	326
Термины	327

В этой главе описано, как объявлять и определять функции. Здесь также обсуждается передача функции аргументов и возвращение из них полученных значений. В языке C++ функции могут быть перегружены, т.е. то же имя может быть использовано для нескольких разных функций. Мы рассмотрим и то, как перегрузить функции, и то, как компилятор выбирает из нескольких перегруженных функций ее соответствующую версию для конкретного вызова. Завершается глава описанием указателей на функции.

Функция (function) — это именованный блок кода. Запуск этого кода на выполнение осуществляется при вызове функции. Функция может получать любое количество аргументов и (обычно) возвращает результат. Функция может быть перегружена, следовательно, то же имя может относиться к нескольким разным функциям.



6.1. Основы функций

Определение функции (function definition) обычно состоит из *типа возвращаемого значения* (return type), имени, списка *параметров* (parameter) и тела функции. Параметры определяются в разделяемом запятыми списке, заключенном в круглые скобки. Выполняемые функцией действия определяются в блоке операторов (см. раздел 5.1, стр. 235), называемом *телом функции* (function body).

Для запуска кода функции используется *оператор вызова* (call operator), представляющий собой пару круглых скобок. Оператор вызова получает выражение, являющееся функцией или указателем на функцию. В круглых скобках располагается разделяемый запятыми список *аргументов* (argument). Аргументы используются для инициализации параметров функции. Тип вызываемого выражения — это тип возвращаемого значения функции.

Создание функции

В качестве примера напомним функцию вычисления факториала заданного числа. Факториал числа n является произведением чисел от 1 до n . Факториал 5, например, равен 120:

$1 * 2 * 3 * 4 * 5 = 120$

Эту функцию можно определить следующим образом:

```
// факториал val равен
// val * (val - 1) * (val - 2) ... * ((val - (val - 1)) * 1)
int fact(int val)
{
    int ret = 1; // локальная переменная для содержания результата по
                // мере его вычисления
    while (val > 1)
        ret *= val--; // присвоение ret произведения ret * val
                    // и декремент val
    return ret;      // вернуть результат
}
```

Функции присвоено имя `fact`. Она получает один параметр типа `int` и возвращает значение типа `int`. В цикле `while` вычисляется факториал с использованием постфиксного оператора декремента (см. раздел 4.5, стр. 204), уменьшающего значение переменной `val` на 1 при каждой итерации. Оператор `return` выполняется в конце функции `fact` и возвращает значение переменной `ret`.

Вызов функции

Чтобы вызвать функцию `fact()`, следует предоставить ей значение типа `int`. Результатом вызова также будет значение типа `int`:

```
int main()
{
    int j = fact(5); // j равно 120, т.е. результату fact(5)
    cout << "5! is " << j << endl;
    return 0;
}
```

Вызов функции осуществляет два действия: он инициализирует параметры функции соответствующими аргументами и передает управление коду этой

функции. При этом выполнение *вызывающей* (calling) функции приостанавливается и начинается выполнение *вызываемой* (called) функции.

Выполнение функции начинается с неявного определения и инициализации ее параметров. Таким образом, когда происходит вызов функции `fact()`, сначала создается переменная типа `int` по имени `val`. Эта переменная инициализируется аргументом, предоставленным при вызове функции `fact()`, которым в данном случае является 5.

Выполнение функции заканчивается оператором `return`. Как и вызов функции, оператор `return` осуществляет два действия: возвращает значение (если оно есть) и передает управление назад *вызывающей* функции. Возвращенное функцией значение используется для инициализации результата вызывающего выражения. Выполнение продолжается с остальной частью выражения, в составе которого осуществлялся вызов. Таким образом, вызов функции `fact()` эквивалентен следующему:

```
int val = 5; // инициализировать val из литерала 5
int ret = 1; // код из тела функции fact
while (val > 1)
    ret *= val--;
int j = ret; // инициализировать j копией ret
```

Параметры и аргументы

Аргументы — это инициализаторы для параметров функции. Первый аргумент инициализирует первый параметр, второй аргумент инициализирует второй параметр и т.д. Хотя порядок инициализации параметров аргументами известен, порядок обработки аргументов не гарантирован (см. раздел 4.1.3, стр. 192). Компилятор может вычислять аргументы в любом порядке по своему предпочтению.

Тип каждого аргумента должен совпадать с типом соответствующего параметра, как и тип любого инициализатора должен совпадать с типом объекта, который он инициализирует. Следует передать точно такое же количество аргументов, сколько у функции параметров. Поскольку каждый вызов гарантированно передаст столько аргументов, сколько у функции параметров, последние всегда будут инициализированы.

Поскольку у функции `fact()` один параметр типа `int`, при каждом ее вызове следует предоставить один аргумент, который может быть преобразован в тип `int` (см. раздел 4.11, стр. 218):

```
fact("hello"); // ошибка: неправильный тип аргумента
fact();        // ошибка: слишком мало аргументов
fact(42, 10, 0); // ошибка: слишком много аргументов
fact(3.14);    // ok: аргумент преобразуется в int
```

Первый вызов терпит неудачу потому, что невозможно преобразование значения типа `const char*` в значение типа `int`. Второй и третий вызовы пе-

редают неправильные количества аргументов. Функцию `fact()` следует вызывать с одним аргументом; ее вызов с любым другим количеством аргументов будет ошибкой. Последний вызов допустим, поскольку значение типа `double` преобразуется в значение типа `int`. В этом случае аргумент неявно преобразуется в тип `int` (с усечением). После преобразования этот вызов эквивалентен следующему:

```
fact(3);
```

Список параметров функции

Список параметров функции может быть пустым, но он не может отсутствовать. При определении функции без параметров обычно используют пустой список параметров. Для совместимости с языком C можно также использовать ключевое слово `void`, чтобы указать на отсутствие параметров:

```
void f1(){ /* ... */ }      // неявно указанный пустой список параметров
void f2(void){ /* ... */ } // явно указанный пустой список параметров
```

Список параметров, как правило, состоит из разделяемого запятыми списка параметров, каждый из которых выглядит как одиночное объявление. Даже когда типы двух параметров одинаковы, объявление следует повторить:

```
int f3(int v1, v2) { /* ... */ }      // ошибка
int f4(int v1, int v2) { /* ... */ } // ok
```

Параметры не могут иметь одинаковые имена. Кроме того, локальные переменные даже в наиболее удаленной области видимости в функции не могут использовать имя, совпадающее с именем любого параметра.

Имена в определении функций не обязательны, но все параметры обычно именуют. Поэтому у каждого параметра обычно есть имя. Иногда у функций есть не используемые параметры. Такие параметры зачастую оставляют безымянными, указывая, что они не используются. Наличие безымянного параметра не изменяет количество аргументов, которые следует передать при вызове. Аргумент при вызове должен быть предоставлен для каждого параметра, даже если он не используется.

Тип возвращаемого значения функции

В качестве типа возвращаемого значения функции применимо большинство типов. В частности, типом возвращаемого значения может быть `void`, это означает, что функция не возвращает значения. Но типом возвращаемого значения не может быть массив (см. раздел 3.5, стр. 161) или функция. Однако функция может вернуть указатель на массив или функцию. Определение функции, возвращающей указатель (или ссылку) на массив, рассматривается в разделе 6.3.3 (стр. 299), а указателя на функцию — в разделе 6.7 (стр. 322).

Упражнения раздела 6.1

Упражнение 6.1. В чем разница между параметром и аргументом?

Упражнение 6.2. Укажите, какие из следующих функций ошибочны и почему. Предложите способ их исправления.

```
(a) int f() {  
    string s;  
    // ...  
    return s;  
}  
(b) f2(int i) { /* ... */ }  
(c) int calc(int v1, int v1) /* ... */ }  
(d) double square(double x) return x * x;
```

Упражнение 6.3. Напишите и проверьте собственную версию функции `fact()`.

Упражнение 6.4. Напишите взаимодействующую с пользователем функцию, которая запрашивает число и вычисляет его факториал. Вызовите эту функцию из функции `main()`.

Упражнение 6.5. Напишите функцию, возвращающую абсолютное значение ее аргумента.



6.1.1. Локальные объекты

В языке C++ имя имеет область видимости (см. раздел 2.2.4, стр. 82), а объекты — *продолжительность существования* (object lifetime). Обе эти концепции важно понимать.

- Область видимости имени — это *часть текста программы*, в которой имя видимо.
- Продолжительность существования объекта — это *время при выполнении программы*, когда объект существует.

Как уже упоминалось, тело функции — это блок операторов. Как обычно, блок формирует новую область видимости, в которой можно определять переменные. Параметры и переменные, определенные в теле функции, называются *локальными переменными* (local variable). Они являются локальными для данной функции и *скрывают* (hide) объявления того же имени во внешней области видимости.

Объекты, определенные вне любой из функций, существуют на протяжении выполнения программы. Такие объекты создаются при запуске программы и не удаляются до ее завершения. Продолжительность существования локальной переменной зависит от того, как она определена.

Автоматические объекты

Объекты, соответствующие обычным локальным переменным, создаются при достижении процессом выполнения определения переменной в функции. Они удаляются, когда процесс выполнения достигает конца блока, в котором определена переменная. Объекты, существующие только во время выполнения блока, известны как *автоматические объекты* (automatic object). После выхода процесса выполнения из блока значения автоматических объектов, созданных в этом блоке, неопределенны.

Параметры — это автоматические объекты. Место для параметров резервируется при запуске функции. Параметры определяются в пределах тела функции. Следовательно, они удаляются по завершении функции.

Автоматические объекты, соответствующие параметрам функции, инициализируются аргументами, переданными функции. Автоматические объекты, соответствующие локальным переменным, инициализируются, если их определение содержит инициализатор. В противном случае они инициализируются значением по умолчанию (см. раздел 2.2.1, стр. 77), а это значит, что значения неинициализированных локальных переменных встроенного типа неопределенны.

Локальные статические объекты

Иногда полезно иметь локальную переменную, продолжительность существования которой не прерывается между вызовами функции. Чтобы получить такие объекты, при определении локальной переменной используют ключевое слово `static`. Каждый *локальный статический объект* (local static object) инициализируется прежде, чем выполнение достигнет определения объекта. Локальная статическая переменная не удаляется по завершении функции; она удаляется по завершении программы.

В качестве простого примера рассмотрим функцию, подсчитывающую количество своих вызовов:

```
size_t count_calls()
{
    static size_t ctr = 0; // значение сохраняется между вызовами
    return ++ctr;
}

int main()
{
    for (size_t i = 0; i != 10; ++i)
        cout << count_calls() << endl;
    return 0;
}
```

Эта программа выводит числа от 1 до 10 включительно.

Прежде чем процесс выполнения впервые достигнет определения переменной `ctr`, она уже будет создана и получит исходное значение 0. Каждый вызов осуществляет инкремент переменной `ctr` и возвращает ее новое значение. При каждом запуске функции `count_calls()` переменная `ctr` уже существует и имеет некое значение, возможно, оставленное последним вызовом функции. Поэтому при втором вызове значением переменной `ctr` будет 1, при третьем — 2 и т.д.

Если у локальной статической переменной нет явного инициализатора, она инициализируется значением по умолчанию (см. раздел 3.3.1, стр. 143), следовательно, локальные статические переменные встроенного типа инициализируются нулем.

Упражнения раздела 6.1.1

Упражнение 6.6. Объясните различия между параметром, локальной переменной и локальной статической переменной. Приведите пример функции, в которой каждая из них могла бы быть полезной.

Упражнение 6.7. Напишите функцию, которая возвращает значение 0 при первом вызове, а при каждом последующем вызове возвращает последовательно увеличивающиеся числа.



6.1.2. Объявление функций

Как и любое другое имя, имя функции должно быть объявлено прежде, чем его можно будет использовать. Подобно переменным (см. раздел 2.2.2, стр. 79), функция может быть определена только однажды, но объявлена может быть многократно. За одним исключением, которое будет описано в разделе 15.3 (стр. 763), можно объявить функцию, которая не определяется до тех пор, пока она не будет использована.

Объявление функции подобно ее определению, но у объявления нет тела функции. В объявлении тело функции заменяет точка с запятой.

Поскольку у объявления функции нет тела, нет никакой необходимости в именах параметров. Поэтому имена параметров зачастую отсутствуют в объявлении. Хотя имена параметров и не обязательны, они зачастую используются, чтобы помочь пользователям функции понять ее назначение:

```
// имена параметров указывают, что итераторы обозначают диапазон
// выводимых значений
void print(vector<int>::const_iterator beg,
          vector<int>::const_iterator end);
```

Эти три элемента объявления (тип возвращаемого значения, имя функции и тип параметров) описывают *интерфейс* (interface) функции. Они задают всю

информацию, необходимую для вызова функции. Объявление функции называют также *прототипом функции* (function prototype).

Объявления функций находятся в файлах заголовка

Напомним, что объявления переменных располагают в файлах заголовка (см. раздел 2.6.3, стр. 116), а определения — в файлах исходного кода. По тем же причинам функции должны быть объявлены в файлах заголовка и определены в файлах исходного кода.

Весьма соблазнительно (и вполне допустимо) размещать объявления функций непосредственно в каждом файле исходного кода, который использует функцию. Однако такой подход утомителен и приводит к ошибкам. Помещая объявления функций в файлы заголовка, можно гарантировать, что все объявления данной функции будут одинаковы. Если необходимо изменить интерфейс функции, достаточно модифицировать его только в одном объявлении.

Рекомендуем

Файл исходного кода, в котором функция *определена*, должен включать заголовок, в котором функция *объявлена*. Так компилятор сможет проверить соответствие определения и объявления.

Упражнения раздела 6.1.2

Упражнение 6.8. Напишите файл заголовка по имени `Chapter6.h`, содержащий объявления функций, написанных для упражнений раздела 6.1 (стр. 271).



6.1.3. Раздельная компиляция

По мере усложнения программ возникает необходимость хранить различные части программы в отдельных файлах. Например, функции, написанные для упражнений раздела 6.1 (стр. 271), можно было бы сохранить в одном файле, а код, использующий их, в других файлах исходного кода. Язык C++ позволяет разделять программы на логические части, предоставляя средство, известное как *раздельная компиляция* (separate compilation). Раздельная компиляция позволяет разделять программы на несколько файлов, каждый из которых может быть откомпилирован независимо.

Компиляция и компоновка нескольких файлов исходного кода

Предположим, например, что определение функции `fact()` находится в файле `fact.cc`, а ее объявление — в файле заголовка `Chapter6.h`. Файл `fact.cc`, как и любой другой файл, использующий эту функцию, будет включать заголовок `Chapter6.h`. Функцию `main()`, вызывающую функцию `fact()`, будем хранить в еще одном файле `factMain.cc`.

Чтобы создать *исполняемый файл* (executable file), следует указать компилятору, где искать весь используемый код. Эти файлы можно было бы откомпилировать следующим образом:

```
$ CC factMain.cc fact.cc # generates factMain.exe or a.out
$ CC factMain.cc fact.cc -o main # generates main or main.exe
```

где `CC` — имя компилятора; `$` — системная подсказка; `#` — начало комментария командной строки. Теперь можно запустить исполняемый файл, который выполнит нашу функцию `main()`.

Если бы изменен был только один из наших файлов исходного кода, то перекомпилировать достаточно было бы только тот файл, который был фактически изменен. Большинство компиляторов предоставляет возможность раздельной компиляции каждого файла. Обычно этот процесс создает файл с расширением `.obj` (на Windows) или `.o` (на UNIX), указывающим, что этот файл содержит *объектный код* (object code).

Компилятор позволяет *скомпоновать* (link) *объектные файлы* (object file) и получить исполняемый файл. На системе авторов раздельная компиляция программы осуществляется следующим образом:

```
$ CC -c factMain.cc # generates factMain.o
$ CC -c fact.cc # generates fact.o
$ CC factMain.o fact.o # generates factMain.exe or a.out
$ CC factMain.o fact.o -o main # generates main or main.exe
```

Сверьтесь с руководством пользователя вашего компилятора, чтобы уточнить, как именно компилировать и запускать программы, состоящие из нескольких файлов исходного кода.

Упражнения раздела 6.1.3

Упражнение 6.9. Напишите собственные версии файлов `fact.cc` и `factMain.cc`. Эти файлы должны включать заголовок `Chapter6.h` из упражнения предыдущего раздела. Используйте эти файлы чтобы понять, как ваш компилятор обеспечивает раздельную компиляцию.



6.2. Передача аргументов

Как уже упоминалось, при каждом вызове функции ее параметры создаются заново. Используемое для инициализации параметра значение предоставляет соответствующий аргумент, переданный при вызове.



Параметры инициализируются точно так же, как и обычные переменные.

Как и у любой другой переменной, взаимодействие параметра и его аргумента определяет тип параметра. Если параметр — ссылка (см. раздел 2.3.1, стр. 85), то параметр привязывается к своему аргументу. В противном случае, значение аргумента копируется.

Когда параметр — ссылка, говорят, что его аргумент *передается по ссылке* (pass by reference) или что функция *вызывается по ссылке* (call by reference). Подобно любой другой ссылке, ссылочный параметр — это только псевдоним объекта, к которому он привязан, т.е. ссылочный параметр — псевдоним своего аргумента.

Когда значение аргумента копируется, параметр и аргумент — независимые объекты. Говорят, что такие аргументы *передаются по значению* (pass by value) или что функция *вызывается по значению* (call by value).



6.2.1. Передача аргумента по значению

При инициализации переменной не ссылочного типа значение инициализатора копируется. Изменения значения переменной никак не влияют на инициализатор:

```
int n = 0; // обычная переменная типа int
int i = n; // i - копия значения переменной n
i = 42;    // значение i изменилось, значение n - нет
```

Передача аргумента по значению осуществляется точно так же; что бы функция не сделала с параметром, на аргумент это не повлияет. Например, в функции `fact()` (см. раздел 6.1, стр. 267) происходит декремент параметра `val`:

```
ret *= val--; // декремент значения val
```

Хотя функция `fact()` изменила значение `val`, это изменение никак не повлияло на переданный ей аргумент. Вызов `fact(i)` не изменяет значение переменной `i`.

Параметры указателя

Указатели (см. раздел 2.3.2, стр. 87) ведут себя, как любой не ссылочный тип. При копировании указателя его значение копируется. После создания копии получается два отдельных указателя. Однако указатель обеспечивает косвенный доступ к объекту, на который он указывает. Значение этого объекта можно изменить при помощи указателя (см. раздел 2.3.2, стр. 90):

```
int n = 0, i = 42;
int *p = &n, *q = &i; // p указывает на n; q указывает на i
*p = 42;               // значение n изменилось, значение p - нет
p = q;                 // теперь p указывает на i; значения i и n
                       // неизменны
```

То же поведение характерно для указателей, являющихся параметрами:

```
// функция получает указатель и обнуляет значение, на которое он
// указывает
void reset(int *ip)
{
    *ip = 0; // изменяет значение объекта, на который указывает ip
    ip = 0;  // изменяет только локальную копию ip; аргумент неизменен
}
```

После вызова функции `reset()` объект, на который указывает аргумент, будет обнулен, но сам аргумент-указатель не изменится:

```
int i = 42;
reset(&i); // изменяет значение i, но не адрес
cout << "i = " << i << endl; // выводит i = 0
```

Рекомендуем

Программисты, привыкшие к языку C, зачастую используют параметры в виде указателей для доступа к объектам вне функции. В языке C++ для этого обычно используют ссылочные параметры.

Упражнения раздела 6.2.1

Упражнение 6.10. Напишите, используя указатели, функцию, меняющую значения двух целых чисел. Проверьте функцию, вызвав ее и отобразив измененные значения.



6.2.2. Передача аргумента по ссылке

Напомним, что операции со ссылками — это фактически операции с объектами, к которым они привязаны (см. раздел 2.3.1, стр. 85):

```
int n = 0, i = 42;
int &r = n; // r привязан к n (т.е. r - другое имя для n)
r = 42;     // теперь n = 42
r = i;      // теперь n имеет то же значение, что и i
i = r;      // i имеет то же значение, что и n
```

Ссылочные параметры используют это поведение. Обычно они применяются, чтобы позволить функции изменить значение одного или нескольких аргументов.

Для примера можно переписать программу `reset` из предыдущего раздела так, чтобы использовать ссылку вместо указателя:

```
// функция, получающая ссылку на объект типа int и обнуляющая его
void reset(int &i) // i - только другое имя объекта, переданного
                  // на обнуление
{
    i = 0; // изменяет значение объекта, на который ссылается i
}
```

Подобно любой другой ссылке, ссылочный параметр связывается непосредственно с объектом, которым он инициализируется. При вызове этой версии функции `reset()` параметр `i` будет связан с любым переданным ей объектом типа `int`. Как и с любой ссылкой, изменения, сделанные с параметром `i`, осуществляются с объектом, на который она ссылается. В данном случае этот объект — аргумент функции `reset()`.

Когда вызывается эта версия функции `reset()`, объект передается непосредственно; поэтому нет никакой необходимости в передаче его адреса:

```
int j = 42;
reset(j); // j передается по ссылке; значение в j изменяется
cout << "j = " << j << endl; // выводит j = 0
```

В этом вызове параметр `i` — это только другое имя переменной `j`. Любое использование параметра `i` в функции `reset()` фактически является использованием переменной `j`.

Использование ссылки во избежание копирования

Копирование объектов больших классов или больших контейнеров снижает эффективность программы. Кроме того, некоторые классы (включая классы ИО) не допускают копирования. Для работы с объектами, тип которых не допускает копирования, функции должны использовать ссылочные параметры.

В качестве примера напишем функцию сравнения длин двух строк. Поскольку строки могут быть очень длинными и их копирования желательно избежать, сделаем параметры ссылками. Так как сравнение двух строк не подразумевает их изменения, сделаем ссылочные параметры константами (см. раздел 2.4.1, стр. 97):

```
// сравнить длины двух строк
bool isShorter(const string &s1, const string &s2)
{
    return s1.size() < s2.size();
}
```

Как будет продемонстрировано в разделе 6.2.3 (стр. 281), для не подлежащих изменению ссылочных параметров функции должны использовать ссылки на константу.

Рекомендуем

Ссылочные параметры, которые не изменяются в функции, должны быть объявлены как `const`.

Использование ссылочных параметров для возвращения дополнительной информации

Функция может вернуть только одно значение. Но что если функции нужно вернуть больше одного значения? Ссылочные параметры позволяют

возвратить несколько результатов. В качестве примера определим функцию `find_char()`, которая возвращает позицию первого вхождения заданного символа в строке. Функция должна также возвращать количество этих символов в строке.

Как же определить функцию, возвращающую и позицию, и количество вхождений? Можно было бы определить новый тип, содержащий позицию и количество. Однако куда проще передать дополнительный ссылочный аргумент, содержащий количество вхождений:

```
// возвращает индекс первого вхождения с в s
// ссылочный параметр occurs содержит количество вхождений
string::size_type find_char(const string &s, char c,
                           string::size_type &occurs)
{
    auto ret = s.size(); // позиция первого вхождения, если оно есть
    occurs = 0;          // установить параметр количества вхождений
    for (decltype(ret) i = 0; i != s.size(); ++i) {
        if (s[i] == c) {
            if (ret == s.size())
                ret = i; // запомнить первое вхождение с
            ++occurs;    // инкремент счетчика вхождений
        }
    }
    return ret; // количество возвращается неявно в параметре occurs
}
```

Когда происходит вызов функции `find_char()`, ей передаются три аргумента: строка, в которой осуществляется поиск, искомый символ и объект типа `size_type` (раздел 3.2.2 стр. 131), содержащий счетчик вхождений. Если `s` является объектом класса `string`, а `ctr` — объектом типа `size_type`, то функцию `find_char()` можно вызвать следующим образом:

```
auto index = find_char(s, 'o', ctr);
```

После вызова значением объекта `ctr` будет количество вхождений символа `o`, а `index` укажет на его первое вхождение, если оно будет. В противном случае значение `index` будет равно `s.size()`, а `ctr` — нулю.

Упражнения раздела 6.2.2

Упражнение 6.11. Напишите и проверьте собственную версию функции `reset()`, получающую ссылку.

Упражнение 6.12. Перепишите программу из упражнения 6.10 раздела 6.2.1 (стр. 277) так, чтобы использовать ссылки вместо указателей при смене значений двух целочисленных переменных. Какая из версий, по вашему, проще в использовании и почему?

Упражнение 6.13. Если `T` — имя типа, объясните различие между функцией, объявленной как `void f(T)` и как `void f(T&)`.

Упражнение 6.14. Приведите пример, когда параметр должен быть ссылочным типом. Приведите пример случая, когда параметр не должен быть ссылкой.

Упражнение 6.15. Объясните смысл каждого из типов параметров функции `find_char()`. В частности, почему `s` — ссылка на константу, а `occurs` — простая ссылка? Почему эти параметры ссылочные, а параметр `c` типа `char` нет? Что будет, сделай мы `s` простой ссылкой? Что если `occurs` сделать константной ссылкой?

6.2.3. Константные параметры и аргументы

При использовании параметров, являющихся константой, следует помнить об обсуждении спецификатора `const` верхнего уровня из раздела 2.4.3 (стр. 101). Как упоминалось в этом разделе, спецификатор `const` верхнего уровня — это тот спецификатор, который относится непосредственно к объекту:

```
const int ci = 42;  // нельзя изменить ci; const верхнего уровня
int i = ci;        // ok: при копировании ci спецификатор const верхнего
                  // уровня игнорируется
int * const p = &i; // const верхнего уровня; нельзя присвоить p
*p = 0;            // ok: изменение при помощи p возможно; i теперь 0
```

Как и при любой другой инициализации, при копировании аргумента для инициализации параметра спецификаторы `const` верхнего уровня игнорируются. В результате спецификатор `const` верхнего уровня для параметров игнорируется. Параметру, у которого есть спецификатор `const` верхнего уровня, можно передать и константный, и неконстантный объект:

```
void fcn(const int i) { /* fcn может читать, но не писать в i */ }
```

Функцию `fcn()` можно вызвать, передав ей аргумент типа `const int` или обычного типа `int`. Тот факт, что спецификаторы `const` верхнего уровня игнорируются у параметра, может иметь удивительные последствия:

```
void fcn(const int i) { /* fcn может читать, но не писать в i */ }
void fcn(int i) { /* ... */ } // ошибка: переопределяет fcn(int)
```

В языке C++ можно определить несколько разных функций с одинаковым именем. Однако это возможно только при достаточно большом различии их списков параметров. Поскольку спецификаторы `const` верхнего уровня игнорируются, мы можем передать те же типы любой версии функции `fcn()`. Вторая версия функции `fcn()` является ошибкой. Несмотря на внешний вид, ее список параметров не отличается от списка первой версии функции `fcn()`.

Параметры в виде указателей или ссылок и константность

Поскольку параметры инициализируются так же, как и переменные, имеет смысл напомнить общие правила инициализации. Можно инициализировать объект со спецификатором `const` нижнего уровня неконстантным объектом, но не наоборот, а простую ссылку следует инициализировать объектом того же типа.

```
int i = 42;
const int *cp = &i; // ok: но cp не может изменить i (раздел 2.4.2,
стр. 99)
const int &r = i;    // ok: но r не может изменить i (раздел 2.4.1,
(стр. 97))
const int &r2 = 42; // ok: (раздел 2.4.1, стр. 97)
int *p = cp;       // ошибка: типы p и cp не совпадают (раздел 2.4.2, стр. 99)
int &r3 = r;        // ошибка: типы r3 и r не совпадают (раздел 2.4.1, стр. 97)
int &r4 = 42; // ошибка: нельзя инициализировать простую ссылку из
                // литерала (раздел 2.3.1, стр. 85)
```

Те же правила инициализации относятся и к передаче параметров:

```
int i = 0;
const int ci = i;
string::size_type ctr = 0;
reset(&i); // вызывает версию функции reset с параметром типа int*
reset(&ci); // ошибка: нельзя инициализировать int* из указателя на
            // объект const int
reset(i);   // вызывает версию функции reset с параметром типа int&
reset(ci);  // ошибка: нельзя привязать простую ссылку к константному
            // объекту ci
reset(42);  // ошибка: нельзя привязать простую ссылку к литералу
reset(ctr); // ошибка: типы не совпадают; ctr имеет беззнаковый тип
// ok: первый параметр find_char является ссылкой на константу
find_char("Hello World!", 'o', ctr);
```

Ссылочную версию функции `reset()` (см. раздел 6.2.2, стр. 277) можно вызывать только для объектов типа `int`. Нельзя передать литерал, выражение, результат которого будет иметь тип `int`, объект, который требует преобразования, или объект типа `const int`. Точно так же версии функции `reset()` с указателем можно передать только объект типа `int*` (см. раздел 6.2.1, стр. 277). С другой стороны, можно передать строковый литерал как первый аргумент функции `find_char()` (см. раздел 6.2.2, стр. 279). Ссылочный параметр этой функции — ссылка на константу, и можно инициализировать ссылку на константу из литералов.



По возможности используйте ссылки на константы

Весьма распространена ошибка, когда не изменяемые функцией параметры определяют как простые ссылки. Это создает у вызывающей стороны функции ложное впечатление, что функция могла бы изменить значение своего аргумента. Кроме того, использование ссылки вместо ссылки на константу

неоправданно ограничивает типы аргументов, применяемые функцией. Как уже упоминалось, нельзя передать константный объект, литерал или требующий преобразования объект как простой ссылочный параметр.

В качестве примера рассмотрим функцию `find_char()` из раздела 6.2.2 (стр. 279). Строковый параметр этой функции правильно сделан ссылкой на константу. Если бы этот параметр был определен как `string&`:

```
// ошибка: первый параметр должен быть const string&
string::size_type find_char(string &s, char c,
string::size_type &occurs);
```

то вызвать ее можно было бы только для объекта класса `string` так:

```
find_char("Hello World", 'o', ctr);
```

Что привело бы к неудаче во времени компиляции.

Более того, эту версию функции `find_char()` нельзя использовать из других функций, которые правильно определяют свои параметры как ссылки на константу. Например, мы могли бы использовать функцию `find_char()` в функции, которая определяет, является ли строка предложением:

```
bool is_sentence(const string &s)
{
    // если в конце s есть точка, то строка s - предложение
    string::size_type ctr = 0;
    return find_char(s, '.', ctr) == s.size() - 1 && ctr == 1;
}
```

Если бы функция `find_char()` получала простую ссылку `string&`, то этот ее вызов привел бы к ошибке при компиляции. Проблема в том, что `s` — ссылка на `const string`, но функция `find_char()` была неправильно определена как получающая простую ссылку.

Было бы заманчиво попытаться исправить эту проблему, изменив тип параметра в функции `is_sentence()`. Но это только распространит ошибку, так как вызывающая сторона функции `is_sentence()` сможет передавать только неконстантные строки.

Правильный способ решения этой проблемы — исправить параметр функции `find_char()`. Если невозможно изменить функцию `find_char()`, определите локальную копию строки `s` в функции `is_sentence()` и передавайте эту строку функции `find_char()`.

Упражнения раздела 6.2.3

Упражнение 6.16. Несмотря на то что следующая функция допустима, она менее полезна, чем могла бы быть. Выявите и исправьте ограничение этой функции:

```
bool is_empty(string& s) { return s.empty(); }
```

Упражнение 6.17. Напишите функцию, определяющую, содержит ли строка какие-нибудь заглавные буквы. Напишите функцию, переводящую всю строку в нижний регистр. Используемые в этих функциях параметры имеют тот же тип? Если да, то почему? Если нет, то тоже почему?

Упражнение 6.18. Напишите объявления для каждой из следующих функций. Написав объявления, используйте имя функции для обозначения того, что она делает.

(a) Функция `compare()` возвращает значение типа `bool` и получает два параметра, являющиеся ссылками на класс `matrix`.

(b) Функция `change_val()` возвращает итератор `vector<int>` и получает два параметра: один типа `int`, а второй итератор для вектора `vector<int>`.

Упражнение 6.19. С учетом следующего объявления определите, какие вызовы допустимы, а какие нет. Объясните, почему они недопустимы.

```
double calc(double);
int count(const string &, char);
int sum(vector<int>::iterator, vector<int>::iterator, int);
vector<int> vec(10);
(a) calc(23.4, 55.1);      (b) count("abcd", 'a');
(c) calc(66);              (d) sum(vec.begin(), vec.end(), 3.8);
```

Упражнение 6.20. Когда ссылочные параметры должны быть ссылками на константу? Что будет, если сделать параметр простой ссылкой, когда это могла быть ссылка на константу?

6.2.4. Параметры в виде массива

Массивы обладают двумя особенностями, влияющими на определение и использование функций, работающих с массивами: массив нельзя скопировать (см. раздел 3.5.1, стр. 162), имя массива при использовании автоматически преобразуется в указатель на его первый элемент (см. раздел 3.5.3, стр. 167). Поскольку копировать массив нельзя, его нельзя передать функции по значению. Так как имя массива автоматически преобразуется в указатель, при передаче массива функции фактически передается указатель на его первый элемент.

Хотя передать массив по значению нельзя, вполне можно написать параметр, который выглядит как массив:

```
// несмотря на внешний вид, эти три объявления функции print эквивалентны
// у каждой функции есть один параметр типа const int*
void print(const int*);
void print(const int[]); // демонстрация намерения получить массив
void print(const int[10]); // размерность только для документирования
```

Независимо от внешнего вида, эти объявления эквивалентны: в каждом объявлена функция с одним параметром типа `const int*`. Когда компилятор

проверяет вызов функции `print()`, он выясняет только то, что типом аргумента является `const int*`:

```
int i = 0, j[2] = {0, 1};
print(&i); // ok: &i - int*
print(j); // ok: j преобразуется в int*, указывающий на j[0]
```

Если передать массив функции `print()`, то этот аргумент автоматически преобразуется в указатель на первый элемент в массиве; размер массива не имеет значения.



Подобно любому коду, который использует массивы, функции, получающие в качестве параметров массив, должны гарантировать невыход за пределы его границ.

Поскольку массивы передаются как указатели, их размер функции обычно неизвестен. Они должны полагаться на дополнительную информацию, предоставляемую вызывающей стороной. Для управления параметрами указателя обычно используются три подхода.

Использование маркера для определения продолжения массива

Первый подход к управлению аргументами в виде массива требует, чтобы массив сам содержал маркер конца. Примером этого подхода являются символьные строки в стиле C (см. раздел 3.5.4, стр. 172). Строки в стиле C хранятся в символьных массивах, последний символ которых является нулевым. Функции, работающие со строками в стиле C, прекращают обработку массива, когда встречают нулевой символ:

```
void print(const char *cp)
{
    if (cp) // если cp не нулевой указатель
        while (*cp) // пока указываемый символ не является нулевым
            cout << *cp++; // вывести символ и перевести указатель
}
```

Это соглашение хорошо работает с данными, где есть очевидное значение конечного маркера (такое, как нулевой символ), который не встречается в обычных данных. Это работает значительно хуже с такими данными, как целые числа, где каждое значение в диапазоне вполне допустимо.

Использование соглашения стандартной библиотеки

Второй подход обычно используется для управления аргументами в виде массива при передаче указателей на первый и следующий после последнего элемент массива. Подобный подход используется в стандартной библиотеке. Подробно этот стиль программирования обсуждается в части II. Используя этот подход, элементы массива можно отобразить следующим образом:

```
void print(const int *beg, const int *end)
{
    // вывести все элементы, начиная с beg и до, но не включая, end
    while (beg != end)
        cout << *beg++ << endl; // вывести текущий элемент
                                // и перевести указатель
}
```

Для вывода текущего элемента и перевода указателя `beg` на следующий элемент массива цикл `while` использует операторы обращения к значению и постфиксного инкремента (см. раздел 4.5, стр. 204). Цикл останавливается, когда `beg` становится равен `end`.

При вызове этой функции передаются два указателя: один на первый подлежащий отображению элемент и один на элемент после последнего:

```
int j[2] = {0, 1};
// j преобразуется в указатель на первый элемент массива j
// второй аргумент - указатель на следующий элемент после конца j
print(begin(j), end(j)); // функции begin и end см. п. 3.5.3 (стр. 168)
```

Эта функция безопасна, пока вызывающая сторона правильно вычисляет указатели. Здесь эти указатели предоставляют библиотечные функции `begin()` и `end()` (см. раздел 3.5.3, стр. 168).

Явная передача параметра размера

Третий подход распространен в программах C и устаревших программах C++. Он подразумевает определение второго параметра, указывающего размер массива. Используя этот подход, перепишем функцию `print()` следующим образом:

```
// const int ia[] - эквивалент const int* ia
// размер передается явно и используется для контроля доступа
// к элементам ia
void print(const int ia[], size_t size)
{
    for (size_t i = 0; i != size; ++i) {
        cout << ia[i] << endl;
    }
}
```

Эта версия использует параметр `size` для определения количества выводимых элементов. Когда происходит вызов функции `print()`, ей следует передать этот дополнительный параметр:

```
int j[] = { 0, 1 }; // массив типа int размером 2
print(j, end(j) - begin(j));
```

Функция безопасна, пока переданный размер не превосходит реальную величину массива.

Параметры массива и константность

Обратите внимание, что все три версии функции `print()` определяли свои параметры массива как указатели на константу. В разделе 6.2.3 (стр. 281) было упомянуто о схожести указателей и ссылок. Когда функция не нуждается в записи элементов массива, параметр массива должен быть указателем на константу (см. раздел 2.4.2, стр. 99). Параметр должен быть простым указателем на неконстантный тип, только если функция должна изменять значения элементов.

Ссылочный параметр массива

Подобно тому, как можно определить переменную, являющуюся ссылкой на массив (см. раздел 3.5.1, стр. 162), можно определить параметр, являющийся ссылкой на массив. Как обычно, ссылочный параметр привязан к соответствующему аргументу, которым в данном случае является массив:

```
// ok: параметр является ссылкой на массив; размерность - часть типа
void print(int (&arr)[10])
{
    for (auto elem : arr)
        cout << elem << endl;
}
```



Круглые скобки вокруг части `&arr` необходимы (см. раздел 3.5.1, стр. 162):

```
f(int &arr[10])    // ошибка: объявляет arr как массив ссылок
f(int (&arr)[10]) // ok: arr - ссылка на массив из десяти целых чисел
```

Поскольку размер массива является частью его типа, на размерность в теле функции вполне можно положиться. Однако тот факт, что размер является частью типа, ограничивает полноценность этой версии функции `print()`. Эту функцию можно вызвать только для массива из десяти целых чисел:

```
int i = 0, j[2] = {0, 1};
int k[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
print(&i); // ошибка: аргумент не массив из десяти целых чисел
print(j); // ошибка: аргумент не массив из десяти целых чисел
print(k); // ok: аргумент массив из десяти целых чисел
```

В разделе 16.1.1 (стр. 824) будет показано, как можно написать эту функцию способом, позволяющим передавать ссылочный параметр массива любого размера.

Передача многомерного массива

Напомним, что в языке C++ нет многомерных массивов (см. раздел 3.6, стр. 177). Вместо многомерных массивов есть массив массивов.

Подобно любому массиву, многомерный массив передается как указатель на его первый элемент (см. раздел 3.6, стр. 180). Поскольку речь идет о массиве

массивов, элемент которого сам является массивом, указатель является указателем на массив. Размер второй размерности (и любой последующий) является частью типа элемента и должен быть определен:

```
// matrix указывает на первый элемент массива, элементы которого являются
// массивами из десяти целых чисел
void print(int (*matrix)[10], int rowSize) { /* ... */ }
```

Объявляет `matrix` указателем на массив из десяти целых чисел.



Круглые скобки вокруг `*matrix` снова необходимы:

```
int *matrix[10]; // массив из десяти указателей
int (*matrix)[10]; // указатель на массив из десяти целых чисел
```

Функцию можно также определить с использованием синтаксиса массива. Как обычно, компилятор игнорирует первую размерность, таким образом, лучше не включать ее:

```
// эквивалентное определение
void print(int matrix[][10], int rowSize) { /* ... */ }
```

Здесь объявление `matrix` выглядит как двумерный массив. Фактически параметр является указателем на массив из десяти целых чисел.

Упражнения раздела 6.2.4

Упражнение 6.21. Напишите функцию, получающую значение типа `int` и указатель на тип `int`, а возвращающую значение типа `int`, если оно больше, или значение, на которое указывает указатель, если больше оно. Какой тип следует использовать для указателя?

Упражнение 6.22. Напишите функцию, меняющую местами два указателя на тип `int`.

Упражнение 6.23. Напишите собственные версии каждой из функций `print()`, представленных в этом разделе. Вызовите каждую из этих функций для вывода `i` и `j`, определенных следующим образом:

```
int i = 0, j[2] = {0, 1};
```

Упражнение 6.24. Объясните поведение следующей функции. Если в коде есть проблемы, объясните, где они и как их исправить.

```
void print(const int ia[10])
{
    for (size_t i = 0; i != 10; ++i)
        cout << ia[i] << endl;
}
```

6.2.5. Функция `main()`: обработка параметров командной строки

Функция `main()` — хороший пример того, как программы на языке C++ передают массивы в функции. До сих пор функция `main()` в примерах определялась с пустым списком параметров.

```
int main() { ... }
```

Но зачастую функции `main()` необходимо передать аргументы. Обычно аргументы функции `main()` используют для того, чтобы позволить пользователю задать набор параметров, влияющих на работу программы. Предположим, например, что функция `main()` программы находится в исполняемом файле по имени `prog`. Параметры программе можно передавать следующим образом:

```
prog -d -o ofile data0
```

Так, параметры командной строки передаются функции `main()` в двух (необязательных) параметрах:

```
int main(int argc, char *argv[]) { ... }
```

Второй параметр, `argv`, является массивом указателей на символьные строки в стиле C, а первый параметр, `argc`, передает количество строк в этом массиве. Поскольку второй параметр является массивом, функцию `main()`, в качестве альтернативы, можно определить следующим образом:

```
int main(int argc, char **argv) { ... }
```

Обратите внимание: указатель `argv` указывает на тип `char*`.

При передаче аргументов функции `main()` первый элемент массива `argv` содержит либо имя программы, либо является пустой строкой. Последующие элементы передают аргументы, предоставленные в командной строке. Элемент сразу за последним указателем гарантированно будет нулем.

С учетом предыдущей командной строки `argc` содержит значение 5, а `argv` — следующие символьные строки в стиле C:

```
argv[0] = "prog"; // может также указывать на пустую строку
argv[1] = "-d";
argv[2] = "-o";
argv[3] = "ofile";
argv[4] = "data0";
argv[5] = 0;
```



ВНИМАНИЕ

При использовании аргументов в массиве `argv` помните, что дополнительные аргументы начинаются с элемента `argv[1]`; элемент `argv[0]` содержит имя программы, а не введенный пользователем параметр.

Упражнения раздела 6.2.5

Упражнение 6.25. Напишите функцию `main()`, получающую два аргумента. Конкатенируйте предоставленные аргументы и выведите полученную строку.

Упражнение 6.26. Напишите программу, которая способна получать параметры командной строки, описанные в этом разделе. Отобразите значения аргументов, переданных функции `main()`.

6.2.6. Функции с переменным количеством параметров

Иногда количество аргументов, подлежащих передаче функции, неизвестно заранее. Например, могла бы понадобиться функция, выводящая сообщения об ошибках, созданные нашей программой. Нам хотелось бы использовать одну функцию, чтобы выводить эти сообщения единообразным способом. Однако различные вызовы этой функции могли бы передавать разные аргументы, соответствующие разным видам сообщений об ошибках.

Новый стандарт предоставляет два основных способа создания функций, получающих переменное количество аргументов: если у всех аргументов тот же тип, можно передать объект библиотечного класса `initializer_list`. Если типы аргументов разные, можно написать функцию специального вида, известную как *шаблон с переменным количеством аргументов* (variadic template), который мы рассмотрим в разделе 16.4 (стр. 878).

В языке C++ есть также специальный тип параметра, многоточие, применяющийся для передачи переменного количества аргументов. Мы кратко рассмотрим параметры в виде многоточия далее в этом разделе. Но следует заметить, что это средство обычно используют только в программах, которые должны взаимодействовать с функциями C.

Параметры типа `initializer_list`

**C++
11** Функцию, получающую произвольное количество аргументов одинакового типа, можно написать, используя параметр типа `initializer_list`. Тип `initializer_list` — это библиотечный класс, который представляет собой массив (см. раздел 3.5, стр. 161) значений определенного типа. Этот тип определен в заголовке `initializer_list`. Операции, предоставляемые классом `initializer_list`, перечислены в табл. 6.1.

Таблица 6.1. Операции, предоставляемые классом `initializer_list`

<code>initializer_list<T> lst;</code>	Инициализация по умолчанию; пустой список элементов типа <code>T</code>
<code>initializer_list<T> lst{a,b,c...};</code>	<code>lst</code> имеет столько элементов, сколько инициализаторов; элементы являются копиями соответствующих инициализаторов. Элементы списка — константы
<code>lst2(lst) lst2 = lst</code>	Копирование или присвоение объекта класса <code>initializer_list</code> не копирует элементы в списке. После копирования первоисточник и копия совместно используют элементы
<code>lst.size()</code>	Количество элементов в списке
<code>lst.begin() lst.end()</code>	Возвращает указатель на первый и следующий после последнего элементы <code>lst</code>

Подобно типу `vector`, тип `initializer_list` является шаблоном (см. раздел 3.3, стр. 141). При определении объекта класса `initializer_list` следует указать тип элементов, которые будет содержать список:

```
initializer_list<string> ls; // initializer_list строк
initializer_list<int> li;   // initializer_list целых чисел
```

В отличие от вектора, элементы списка `initializer_list` всегда константы; нет никакого способа изменить значение его элементов.

Функцию отображения сообщений об ошибках с переменным количеством аргументов можно написать следующим образом:

```
void error_msg(initializer_list<string> il)
{
    for (auto beg = il.begin(); beg != il.end(); ++beg)
        cout << *beg << " ";
    cout << endl;
}
```

Методы `begin()` и `end()` объектов класса `initializer_list` аналогичны таковым у класса `vector` (см. раздел 3.4.1, стр. 153). Метод `begin()` предоставляет указатель на первый элемент списка, а метод `end()` — на следующий элемент после последнего. Наша функция инициализирует переменную `beg` указателем на первый элемент и перебирает все элементы списка `initializer_list`. В теле цикла осуществляется обращение к значению `beg`, что позволяет получить доступ к текущему элементу и вывести его значение.

При передаче последовательности значений в параметре типа `initializer_list` последовательность следует заключить в фигурные скобки:

```
// expected и actual - строки
if (expected != actual)
    error_msg({"functionX", expected, actual});
else
    error_msg({"functionX", "okay"});
```

Здесь той же функции `error_msg()` передаются при первом вызове три значения, а при втором — два.

У функции с параметром `initializer_list` могут быть также и другие параметры. Например, у нашей системы отладки мог бы быть класс `ErrCode`, представляющий различные виды ошибок. Мы можем пересмотреть свою программу так, чтобы в дополнение к списку `initializer_list` передавать параметр типа `ErrCode` следующим образом:

```
void error_msg(ErrCode e, initializer_list<string> il)
{
    cout << e.msg() << ": ";
    for (const auto &elem : il)
        cout << elem << " ";
    cout << endl;
}
```

Поскольку класс `initializer_list` имеет члены `begin()` и `end()`, мы можем использовать для обработки элементов серийный оператор `for` (см. раздел 5.4.3, стр. 251). Эта программа, как и предыдущая версия, перебирает элементы заключенного в фигурные скобки списка значений, переданных параметру `il`.

Для этой версии необходимо пересмотреть вызовы так, чтобы передать аргумент типа `ErrCode`:

```
if (expected != actual)
    error_msg(ErrCode(42), {"functionX", expected, actual});
else
    error_msg(ErrCode(0), {"functionX", "okay"});
```



Параметр в виде многоточия

Параметры в виде многоточия предоставляются языком C++ для взаимодействия программ с кодом на языке C, использующим такое средство библиотеки C, как `varargs`. В других целях параметр в виде многоточия не следует использовать. Использование `varargs` описано в документации компилятора C.



Параметры в виде многоточия должны использоваться только для таких типов, которые есть и у языка C, и у C++. В частности, большинство объектов типа класса копируются неправильно, когда передаются параметру в виде многоточия.

Параметр в виде многоточия может быть только последним элементом в списке параметров и может принять любую из двух форм:

```
void foo(parm_list, ...);
void foo(...);
```

Первая форма определяет тип (типы) для нескольких параметров функции `foo()`. Контроль типов аргументов, соответствующих определенным параметрам, осуществляется как обычно. Для аргументов, соответствующих параметру в виде многоточия, никакого контроля типов нет. В первой форме запятая после объявления параметра необязательна.

Упражнения раздела 6.2.6

Упражнение 6.27. Напишите функцию, получающую параметр типа `initializer_list<int>` и возвращающую сумму элементов списка.

Упражнение 6.28. Во второй версии функции `error_msg()`, где у нее есть параметр типа `ErrCode`, каков тип элемента в цикле `for`?

Упражнение 6.29. При использовании типа `initializer_list` в серийном операторе `for` использовали бы вы ссылку как управляющую переменную цикла? Объясните почему.

6.3. Типы возвращаемого значения и оператор `return`

Оператор `return` завершает выполнение функции и возвращает управление той функции, которая вызвала текущую. Существуют две формы оператора `return`:

```
return;
return выражение;
```



6.3.1. Функции без возвращаемого значения

Оператор `return` без значения применим только в такой функции, типом возвращаемого значения которой объявлен `void`. Функции, возвращаемым типом которых объявлен `void`, необязательно должны содержать оператор `return`. В функции типа `void` оператор `return` неявно размещается после последнего оператора.

Как правило, функции типа `void` используют оператор `return` для преждевременного завершения выполнения. Это аналогично использованию оператора `break` (см. раздел 5.5.1, стр. 254) для выхода из цикла. Например, можно написать функцию `swap()`, которая не делает ничего, если значения идентичны:

```
void swap(int &v1, int &v2)
{
    // если значения равны, их замена не нужна; можно выйти сразу
```

```

if (v1 == v2)
    return;
// если мы здесь, придется поработать
int tmp = v2;
v2 = v1;
v1 = tmp;
// явно указывать оператор return не обязательно
}

```

Сначала эта функция проверяет, не равны ли значения, и если это так, то завершает работу. Если значения не равны, функция меняет их местами. После последнего оператора присвоения осуществляется неявный выход из функции.

Функции, для возвращаемого значения которых указан тип `void`, вторую форму оператора `return` могут использовать только для возвращения результата вызова другой функции, которая возвращает тип `void`. Возвращение любого другого выражения из функции типа `void` приведет к ошибке при компиляции.



6.3.2. Функции, возвращающие значение

Вторая форма оператора `return` предназначена для возвращения результата из функции. Каждый случай возвращения значения типа, отличного от `void`, должен вернуть значение. Возвращаемое значение должно иметь тип, либо совпадающий, либо допускающий неявное преобразование (см. раздел 4.11, стр. 218) в тип, указанный для возвращаемого значения функции при определении.

Хотя язык C++ не может гарантировать правильность результата, он способен гарантировать, что каждое возвращаемое функцией значение будет соответствовать объявленному типу. Это может получиться не во всех случаях, компилятор попытается обеспечить возвращение значения и выход только через допустимый оператор `return`. Например:

```

// некорректное возвращение значения, этот код не будет откомпилирован
bool str_subrange(const string &str1, const string &str2)
{
    // размеры одинаковы: вернуть обычный результат сравнения
    if (str1.size() == str2.size())
        return str1 == str2; // ok: == возвращает bool
    // найти размер меньшей строки; условный оператор см. раздел 4.7,
    // стр. 208
    auto size = (str1.size() < str2.size())
        ? str1.size() : str2.size();
    // просмотреть все элементы до размера меньшей строки
    for (decltype(size) i = 0; i != size; ++i) {
        if (str1[i] != str2[i])
            return; // ошибка #1: нет возвращаемого значения; компилятор
                // должен обнаружить эту ошибку
    }
}

```

```

}
// ошибка #2: выполнение может дойти до конца функции, так и
// не встретив оператор return
// компилятор может и не обнаружить эту ошибку
}

```

Оператор `return` в цикле `for` является ошибочным потому, что он не в состоянии вернуть значение. Эту ошибку компилятор должен обнаружить.

Вторая ошибка заключается в том, что функция не имеет оператора `return` после цикла. Если произойдет вызов этой функции со строкой, являющейся подмножеством другой, процесс выполнения минует цикл `for`. Однако оператор `return` для этого случая не предусмотрен. Эту ошибку компилятор может и не обнаружить. В этом случае поведение программы во время выполнения будет непредсказуемо.



Отсутствие оператора `return` после цикла, который этот оператор содержит, является особенно коварной ошибкой. Однако большинство компиляторов ее не обнаружит.

Как возвращаются значения

Значения функций возвращаются тем же способом, каким инициализируются переменные и параметры: возвращаемое значение используется для инициализации временного объекта в точке вызова, и этот временный объект является результатом вызова функции.

В функциях, возвращающих локальные переменные, важно иметь в виду правила инициализации. Например, можно написать функцию, которой передают счетчик, слово и окончание. Функция возвращает множественную форму слова, если счетчик больше 1:

```

// вернуть множественную форму слова, если ctr больше 1
string make_plural(size_t ctr, const string &word,
                  const string &ending)
{
    return (ctr > 1) ? word + ending : word;
}

```

Тип возвращаемого значения этой функции — `string`, это значит, что возвращаемое значение копируется в точке вызова. Функция возвращает копию значения `word` или безымянную временную строку, полученную конкатенацией `word` и `ending`.

Когда функция возвращает ссылку, она, подобно любой другой ссылке, является только другим именем для объекта, на который ссылается. Рассмотрим, например, функцию, возвращающую ссылку на более короткую из двух переданных ей строк:

```

// вернуть ссылку на строку, которая короче
const string &shorterString(const string &s1, const string &s2)

```



```
{  
    return s1.size() <= s2.size() ? s1 : s2;  
}
```

Параметры и возвращаемое значение имеют тип ссылки на `const string`. Строки не копируются ни при вызове функции, ни при возвращении результата.

Никогда не возвращайте ссылку на локальный объект

По завершении работы функции все хранилища ее локальных объектов освобождаются (см. раздел 6.1.1, стр. 271). Поэтому после завершения работы функции ссылки на ее локальные объекты ссылаются на несуществующие объекты.

```
// катастрофа: функция возвращает ссылку на локальный объект  
const string &manip()  
{  
    string ret;  
    // обработать ret некоторым образом  
    if (!ret.empty())  
        return ret; // ошибка: возвращение ссылки на локальный объект!  
    else  
        return "Empty"; // ошибка: "Empty" - локальная временная строка  
}
```

Эта функция приведет к отказу во время выполнения, поскольку она возвращает ссылку на локальный объект. Когда функция завершит работу, область памяти, которую занимала переменная `ret`, будет освобождена. Возвращаемое значение будет ссылаться на ту область памяти, которая уже недоступна.

Оба оператора `return` возвращают здесь неопределенное значение — неизвестно, что будет, попробуй мы использовать значение, возвращенное функцией `manip()`. В первом операторе `return` очевидно, что функция пытается вернуть ссылку на локальный объект. Во втором случае строковый литерал преобразуется в локальный временный объект класса `string`. Этот объект, как и строка `s`, является локальным объектом функции `manip()`. Область памяти, в которой располагается временный объект, освобождается по завершении функции. Оба оператора `return` возвращают ссылки на области памяти, которые больше недоступны.



Чтобы удостовериться в безопасности возвращения значения, следует задаться вопросом: к какому существовавшему ранее объекту относится ссылка?

Возвращать указатель на локальный объект нельзя по тем же причинам. По завершении функции локальные объекты освобождаются, и указатель указывает на несуществующий объект.

Функции, возвращающие типы класса и оператор вызова

Подобно любому оператору, оператор вызова обладает порядком (ассоциативностью) и приоритетом (см. раздел 4.1.2, стр. 190). У оператора вызова тот же приоритет, что и у операторов “точка” и “стрелка” (см. раздел 4.6, стр. 207). Как и эти операторы, оператор вызова имеет левосторонний порядок. В результате, если функция возвращает указатель, ссылку или объект типа класса, результат вызова можно использовать для обращения к члену полученного объекта.

Например, размер более короткой строки можно определить следующим образом:

```
// обращение к методу size объекта строки, возвращенной shorterString
auto sz = shorterString(s1, s2).size();
```

Поскольку эти операторы имеют левосторонний порядок, результат вызова функции `shorterString()` будет левым операндом точечного оператора. Этот оператор выбирает метод `size()` объекта строки. Этот метод становится левым операндом второго оператора вызова.

Возвращаемая ссылка является l-значением

Является ли вызов функции l-значением (см. раздел 4.1.1, стр. 189), зависит от типа возвращаемого значения функции. Вызовы функции, возвращающей ссылку, являются l-значением; другие типы возвращаемого значения являются r-значениями. Вызов функции, возвращающей ссылку, применяется таким же способом, как и любое другое l-значение. В частности, можно осуществлять присвоение результату вызова функции, возвращающей ссылку на неконстанту:

```
char &get_val(string &str, string::size_type ix)
{
    return str[ix]; // get_val подразумевает, что данный индекс допустим
}

int main()
{
    string s("a value");
    cout << s << endl; // отображает значение
    get_val(s, 0) = 'A'; // изменяет s[0] на A
    cout << s << endl; // отображает значение A
    return 0;
}
```

Может быть несколько странно видеть вызов функции слева от оператора присвоения. Однако в этом нет ничего необычного. Возвращаемое значение — ссылка, поэтому вызов — это l-значение, а любое l-значение может быть левым операндом оператора присвоения.

Если тип возвращаемого значения является ссылкой на константу, то (как обычно) присвоение результату вызова невозможно:

```
shorterString("hi", "bye") = "X"; // ошибка: возвращаемое значение
// является константой
```

Списочная инициализация возвращаемого значения

**C++
11** По новому стандарту функции могут возвращать окруженный скобками список значений. Подобно любому другому случаю возвращения значения, список используется для инициализации временного объекта, представляющего возвращение функцией значения. Если список пуст, временный объект инициализируется значением по умолчанию (см. раздел 3.3.1, стр. 143). В противном случае возвращаемое значение зависит от типа возвращаемого значения функции.

Для примера вернемся к функции `error_msg` из раздел 6.2.6 (стр. 290). Эта функция получала переменное количество строковых аргументов и выводило сообщение об ошибке, составленное из переданных строк. Теперь вместо вызова функции `error_msg()` мы возвратим вектор, содержащий строки сообщений об ошибке:

```
vector<string> process()
{
    // ...
    // expected и actual - строки
    if (expected.empty())
        return {}; // вернуть пустой вектор
    else if (expected == actual)
        return {"functionX", "okay"}; // вернуть вектор
                                     // инициализированный списком
    else
        return {"functionX", expected, actual};
}
```

В первом операторе `return` возвращается пустой список. В данном случае возвращенный обработанный вектор будет пуст. В противном случае возвращается вектор, инициализированный двумя или тремя элементами, в зависимости от того, равны ли `expected` и `actual`.

У функции, возвращающей встроенный тип, заключенный в скобки список может содержать хотя бы одно значение, и это значение не должно требовать сужающего преобразования (см. раздел 2.2.1, стр. 77). Если функция возвращает тип класса, то используемые инициализаторы определяет сам класс (см. раздел 3.3.1, стр. 142).

Возвращение значения из функции `main()`

Есть одно исключение из правила, согласно которому функция с типом возвращаемого значения, отличного от `void`, обязана вернуть значение: функция `main()` может завершить работу без возвращения значения. Если процесс выполнения достигает конца функции `main()` и нет никакого значения для возвращения, компилятор неявно добавляет возвращение значения 0.

Как упоминалось в разделе 1.1 (стр. 26), значение, возвращаемое из функции `main()`, рассматривается как индикатор состояния. Возвращение нулевого значения означает успех; большинство других значений — неудачу. У значения, отличного от нуля, есть машинно-зависимое значение. Чтобы сделать его независимым от машины, заголовок `stdlib` определяет две переменные препроцессора (см. раздел 2.3.2, стр. 89), которые можно использовать для индикации успеха или отказа:

```
int main()
{
    if (some_failure)
        return EXIT_FAILURE; // определено в stdlib
    else
        return EXIT_SUCCESS; // определено в stdlib
}
```

Поскольку это переменные препроцессора, им не должна предшествовать часть `std::` и их нельзя использовать в объявлениях `using`.

Рекурсия

Функция, которая вызывает себя прямо или косвенно, является *рекурсивной функцией* (recursive function). В качестве примера можно переписать функцию вычисления факториала так, чтобы использовать рекурсию:

```
// вычислить val!, т.е. 1 * 2 * 3 ... * val
int factorial(int val)
{
    if (val > 1)
        return factorial(val-1) * val;
    return 1;
}
```

В этой реализации осуществляется рекурсивный вызов функции `factorial()`, чтобы вычислить факториал числа, начиная со значения, первоначально переданного `val`, и далее в обратном порядке. Когда значение `val` достигнет 1, рекурсия останавливается и возвращается значение 1.

В рекурсивной функции всегда должно быть определено *условие выхода* или *останова* (stopping condition); в противном случае рекурсия станет бесконечной, т.е. функция продолжит вызывать себя до тех пор, пока стек программы не будет исчерпан. Иногда эта ошибка называется *бесконечной рекурсией* (infinite recursion). В случае функции `factorial()` условием выхода является равенство значения параметра `val` единице.

Ниже приведена трассировка выполнения функции `factorial()` при передаче ей значения 5.

Трассировка вызова функции factorial (5)

Вызов	Возвращает	Значение
factorial(5)	factorial(4) * 5	120
factorial(4)	factorial(3) * 4	24
factorial(3)	factorial(2) * 3	6
factorial(2)	factorial(1) * 2	2
factorial(1)	1	1



Функция `main()` не может вызывать сама себя.

Упражнения раздела 6.3.2

Упражнение 6.30. Откомпилируйте версию функции `str_subrange()`, представленной на стр. 293, и посмотрите, что ваш компилятор делает с указанными сообщениями об ошибках.

Упражнение 6.31. Когда допустимо возвращение ссылки? Когда ссылки на константу?

Упражнение 6.32. Укажите, корректна ли следующая функция. Если да, то объясните, что она делает; в противном случае исправьте ошибки, а затем объясните все.

```
int &get(int *array, int index) { return array[index]; }
int main() {
    int ia[10];
    for (int i = 0; i != 10; ++i)
        get(ia, i) = i;
}
```

Упражнение 6.33. Напишите рекурсивную функцию, выводящую содержимое вектора.

Упражнение 6.34. Что случится, если условие остановки функции `factorial()` будет таким:

```
if (val != 0)
```

Упражнение 6.35. Почему в вызове функции `factorial()` мы передали `val-1`, а не `val--`?

6.3.3. Возвращение указателя на массив

Поскольку копировать массив нельзя, функция не может вернуть его. Но функция может вернуть указатель или ссылку на массив (см. раз-

дел 3.5.1, стр. 162). К сожалению, синтаксис, обычно используемый для определения функций, которые возвращают указатели или ссылки на массив, довольно сложен. К счастью, такие объявления можно упростить. Например, можно использовать псевдоним типа (см. раздел 2.5.1, стр. 106):

```
typedef int arrT[10]; // arrT синоним для типа массива из десяти
                      // целых чисел
using arrtT = int[10]; // эквивалентное объявление arrT;
                      // см. раздел 2.5.1, стр. 106
arrT* func(int i);    // func возвращает указатель на массив из
                      // пяти целых чисел
```

где `arrT` — это синоним для массива из десяти целых чисел. Поскольку нельзя вернуть массив, мы определяем тип возвращаемого значения как указатель на этот тип. Таким образом, функция `func()` получает один аргумент типа `int` и возвращает указатель на массив из десяти целых чисел.

Объявление функции, возвращающей указатель на массив

Чтобы объявить функцию `func()`, не используя псевдоним типа, следует вспомнить, что размерность массива следует за определяемым именем:

```
int arr[10];           // arr массив из десяти целых чисел
int *p1[10];          // p1 массив из десяти указателей
int (*p2)[10] = &arr; // p2 указывает на массив из десяти целых чисел
```

Подобно этим объявлениям, если необходимо определить функцию, которая возвращает указатель на массив, размерность должна следовать за именем функции. Однако функция имеет список параметров, который также следует за именем. Список параметров предшествует размерности. Следовательно, функция, которая возвращает указатель на массив, имеет такую форму:

*Тип (*функция(список_параметров)) [размерность]*

Как и в любом другом объявлении массива, *Тип* — это тип элементов, а *размерность* — это размер массива. Круглые скобки вокруг части *(*функция (список_параметров))* необходимы по той же причине, по которой они были нужны при определении указателя `p2`. Без них мы определили бы функцию, которая возвращает массив указателей.

В качестве конкретного примера рассмотрим следующее объявление функции `func()`, не использующей псевдоним типа:

```
int (*func(int i))[10];
```

Чтобы понять это объявление, имеет смысл прочитать его следующим образом:

- `func(int)` указывает, что функцию `func()` можно вызвать с аргументом типа `int`;

- `(*func(int))` указывает, что можно обратиться к значению результата этого вызова;
- `(*func(int))[10]` указывает, что обращение к значению результата вызова функции `func()` возвращает массив из десяти элементов;
- `int (*func(int))[10]` указывает, что типом элементов этого массива является `int`.

Использование замыкающего типа возвращаемого значения

**C++
11**

По новому стандарту есть и другой способ упростить объявления функции `func()` — с использованием *замыкающего типа возвращаемого значения* (trailing return type). Оно может быть определено для любой функции, но полезней всего оно для функций со сложными типами возвращаемого значения, такими как указатели (или ссылки) на массивы. Замыкающий тип возвращаемого значения следует за списком параметров и предваряется символом `->`. Чтобы сообщить о том, что возвращаемое значение следует за списком параметров, ключевое слово `auto` располагается там, где обычно присутствует тип возвращаемого значения:

```
// fcn получает аргумент типа int и возвращает указатель на массив
// из десяти целых чисел
auto func(int i) -> int(*)[10];
```

Поскольку тип возвращаемого значения указан после списка параметров, проще заметить, что функция `func()` возвращает указатель и что этот указатель указывает на массив из десяти целых чисел.

Использование спецификатора `decltype`

**C++
11**

В качестве другой альтернативы, если известен массив (массивы), указатель на который способна вернуть наша функция, можно использовать спецификатор `decltype`, чтобы объявить тип возвращаемого значения. Например, следующая функция возвращает указатель на один из двух массивов, в зависимости от значения ее параметра:

```
int odd[] = {1,3,5,7,9};
int even[] = {0,2,4,6,8};
// возвращает указатель на массив из пяти элементов типа int
decltype(odd) *arrPtr(int i)
{
    return (i % 2) ? &odd : &even; // возвращает указатель на массив
}
```

Тип возвращаемого значения функции `arrPtr()` указан как `decltype`, свидетельствуя о том, что функция возвращает указатель на любой тип, который имеет `odd`. В данном случае этот объект является массивом, поэтому функция `arrPtr()` возвращает указатель на массив из пяти целых чисел.

Единственная сложность здесь в том, что следует помнить, что спецификатор `decltype` не преобразовывает автоматически массив в указатель соответствующего ему типа. Тип, возвращенный спецификатором `decltype`, является типом массива, для которого нужно добавить `*`, чтобы указать, что функция `arrPtr()` возвращает указатель.

Упражнения раздела 6.3.3

Упражнение 6.36. Напишите объявление функции, возвращающей ссылку на массив из десяти строк, не используя ни замыкающий тип возвращаемого значения, ни спецификатор `decltype` или псевдоним типа.

Упражнение 6.37. Напишите три дополнительных объявления для функций предыдущего упражнения. Нужно использовать псевдоним типа, замыкающий тип возвращаемого значения и спецификатор `decltype`. Какую форму вы предпочитаете и почему?

Упражнение 6.38. Перепишите функцию `arrPtr()` так, чтобы она возвращала ссылку на массив.

6.4. Перегруженные функции

Функции, расположенные в одной области видимости, называются *перегруженными* (overloaded), если они имеют одинаковые имена, но разные списки параметров. Пример определения нескольких функций по имени `print()` приведен в разделе 6.2.4 (стр. 283):

```
void print(const char *cp);
void print(const int *beg, const int *end);
void print(const int ia[], size_t size);
```

Эти функции выполняют одинаковое действие, но их параметры относятся к разным типам. При вызове такой функции компилятор принимает решение о применении конкретной версии на основании типа переданного аргумента:

```
int j[2] = {0,1};
print("Hello World");           // вызов print(const char*)
print(j, end(j) - begin(j));    // вызов print(const int*, size_t)
print(begin(j), end(j));        // вызов print(const int*, const int*)
```

Перегрузка функций избавляет от необходимости придумывать (и помнить) имена, существующие только для того, чтобы помочь компилятору выяснить, которую из функций применять при вызове.



Функция `main()` не может быть перегружена.

Определение перегруженных функций

Рассмотрим приложение базы данных с несколькими функциями для поиска записи на основании имени, номера телефона, номер счета и т.д. Перегрузка функций позволит определить коллекцию функций, каждая по имени `lookup()`, которые отличаются тем, как они осуществляют поиск. Мы сможем вызвать функцию `lookup()`, передав значение любого из следующих типов:

```
Record lookup(const Account&); // поиск по счету
Record lookup(const Phone&);  // поиск по телефону
Record lookup(const Name&);    // поиск по имени
Account acct;
Phone phone;
Record r1 = lookup(acct); // вызов версии, получающей Account
Record r2 = lookup(phone); // вызов версии, получающей Phone
```

Здесь у всех трех функций одинаковое имя, но все же это три разные функции. Чтобы выяснить, которую из них вызвать, компилятор использует тип (типы) аргументов.

Перегруженные функции должны отличаться по количеству или типу (типам) своих параметров. Каждая из функций выше получает один параметр, но типы у этих параметров разные.

Функции не могут отличаться только типами возвращаемого значения. Если списки параметров функций совпадают, а типы возвращаемого значения отличаются, то это будет ошибкой:

```
Record lookup(const Account&);
bool lookup(const Account&); // ошибка: отличается только типом
                             // возвращаемого значения
```

Различие типов параметров

Два списка параметров могут быть идентичными, даже если они не выглядят одинаково:

```
// каждая пара объявляет ту же функцию
Record lookup(const Account &acct);
Record lookup(const Account&); // имена параметров игнорируются
typedef Phone Telno;
Record lookup(const Phone&);
Record lookup(const Telno&); // Telno и Phone того же типа
```

Первое объявление в первой паре именует свой параметр. Имена параметров предназначены только для документирования. Они не изменяют список параметров.

Во второй паре типы только выглядят разными, `Telno` — не новый тип, это только синоним типа `Phone`. Псевдоним типа (см. раздел 2.5.1, стр. 106) предоставляет альтернативное имя для уже существующего типа, а не создает новый тип. Поэтому два параметра, отличающиеся только тем, что один использует имя типа, а другой его псевдоним, не являются разными.



Перегрузка и константные параметры

Как упоминалось в разделе 6.2.3 (стр. 280), спецификатор `const` верхнего уровня (см. раздел 2.4.3, стр. 101) никак не влияет на объекты, которые могут быть переданы функции. Параметр, у которого есть спецификатор `const` верхнего уровня, неотличим от такового без спецификатора `const` верхнего уровня:

```
Record lookup(Phone);
Record lookup(const Phone); // повторно объявляет Record lookup(Phone)
Record lookup(Phone*);
Record lookup(Phone* const); // повторно объявляет Record lookup(Phone*)
```

Здесь вторые объявления повторно объявляет ту же функцию, что и первые.

С другой стороны, функцию можно перегрузить на основании того, является ли параметр ссылкой (или указателем) на константную или неконстантную версию того же типа; речь идет о спецификаторе `const` нижнего уровня:

```
// функции, получающие константную и неконстантную ссылку (или
// указатель), имеют разные параметры
Record lookup(Account&); // функция получает ссылку на Account
Record lookup(const Account&); // новая функция получает константную
// ссылку
Record lookup(Account*); // новая функция получает указатель
// на Account
Record lookup(const Account*); // новая функция получает указатель на
// константу
```

В этих случаях компилятор может использовать константность аргумента, чтобы различить, какую функцию применять. Поскольку нет преобразования (см. раздел 4.11.2, стр. 222) из константы, можно передать константный объект (или указатель на константу) только версии с константным параметром. Так как преобразование в константу возможно, можно вызвать функцию и неконстантного объекта, и указателя на неконстантный объект. Однако, как будет представлено в разделе 6.6.1 (стр. 321), компилятор предпочтет неконстантные версии при передаче неконстантного объекта или указателя на неконстантный объект.

СОВЕТ. КОГДА НЕ СЛЕДУЕТ ПЕРЕГРУЖАТЬ ФУНКЦИИ

Хотя перегрузка функций позволяет избежать необходимости создавать и запоминать имена общепринятых операций, она не всегда целесообразна. В некоторых случаях разные имена функций предоставляют дополнительную информацию, которая упрощает понимание программы. Давайте рассмотрим набор функций-членов класса `Screen`, отвечающих за перемещение курсора.

```
Screen& moveHome();
Screen& moveAbs(int, int);
Screen& moveRel(int, int, string direction);
```

На первый взгляд может показаться, что этот набор функций имеет смысл перегрузить под именем `move`:

```
Screen& move();
Screen& move(int, int);
Screen& move(int, int, string direction);
```

Однако при перегрузке этих функций мы потеряли информацию, которая была унаследована именами функции. Хотя перемещение курсора — это общая операция, совместно используемая всеми этими функциями, специфический характер перемещения уникален для каждой из этих функций. Рассмотрим, например, функцию `moveHome()`, осуществляющую вполне определенное перемещение курсора. Какое из двух приведенных ниже обращений понятнее при чтении кода?

```
// которая из записей понятней?
myScreen.moveHome(); // вероятно, эта!
myScreen.move();
```

Оператор `const_cast` и перегрузка

В разделе 4.11.3 (стр. 224) упоминалось, что оператор `const_cast` особенно полезен в контексте перегруженных функций. В качестве примера вернемся к функции `shorterString()` из раздела 6.3.2 (стр. 294):

```
// вернуть ссылку на строку, которая короче
const string &shorterString(const string &s1, const string &s2)
{
    return s1.size() <= s2.size() ? s1 : s2;
}
```

Эта функция получает и возвращает ссылки на константную строку. Мы можем вызвать функцию с двумя неконстантными строковыми аргументами, но как результат получим ссылку на константную строку. Могла бы понадобиться версия функции `shorterString()`, которая, получив неконстантные аргументы, возвратит обычную ссылку. Мы можем написать эту версию функции, используя оператор `const_cast`:

```
string &shorterString(string &s1, string &s2)
{
    auto &r = shorterString(const_cast<const string&>(s1),
                           const_cast<const string&>(s2));
    return const_cast<string&>(r);
}
```

Эта версия вызывает константную версию функции `shorterString()` при приведении типов ее аргументов к ссылкам на константу. Функция возвращает ссылку на тип `const string`, которая, как известно, привязана к одному из исходных, неконстантных аргументов. Следовательно, приведение этой строки назад к обычной ссылке `string&` при возвращении вполне безопасно.

Вызов перегруженной функции

Когда набор перегруженных функций определен, необходима возможность вызвать их с соответствующими аргументами. *Подбор функции* (function matching), известный также как *поиск перегруженной функции* (overload resolution), — это процесс, в ходе которого вызов функции ассоциируется с определенной версией из набора перегруженных функций. Компилятор определяет, какую именно версию функции использовать при вызове, сравнивая аргументы вызова с параметрами каждой функции в наборе.

Как правило, вовсе несложно выяснить, допустим ли вызов, и если он допустим, то какая из версий функции будет использована компилятором. Функции в наборе перегруженных версий отличаются количеством или типом аргументов. В таких случаях определить используемую функцию просто. Подбор функции усложняется в случае, когда количество параметров одинаково и они допускают преобразование (см. раздел 4.11, стр. 218) переданных аргументов. Распознавание вызовов компилятором при наличии преобразований рассматривается в разделе 6.6 (стр. 317), а пока следует понять, что при любом вызове перегруженной функции возможен один из трех результатов.

- Компилятор находит одну функцию, которая является *наилучшим соответствием* (best match) для фактических аргументов, и создает код ее вызова.
- Компилятор не может найти ни одной функции, параметры которой соответствуют аргументам вызова. В этом случае компилятор сообщает об ошибке *отсутствия соответствия* (no match).
- Компилятор находит несколько функций, которые в принципе подходят, но ни одна из них не соответствует полностью. В этом случае компилятор также сообщает об ошибке, об ошибке *неоднозначности вызова* (ambiguous call).

Упражнения раздела 6.4

Упражнение 6.39. Объясните результат второго объявления в каждом из следующих наборов. Укажите, какое из них (если есть) недопустимо.

- (a) `int calc(int, int);`
 `int calc(const int, const int);`
- (b) `int get();`
 `double get();`
- (c) `int *reset(int *);`
 `double *reset(double *);`



6.4.1. Перегрузка и область видимости



Обычно объявлять функцию локально нежелательно. Но чтобы объяснить, как область видимости взаимодействует с перегрузкой, мы будем нарушать это правило и используем локальные объявления функций.

Новички в программировании на языке C++ зачастую не понимают взаимодействия между областью видимости и перегрузкой. Однако у перегрузки нет никаких специальных свойств относительно области видимости. Как обычно, если имя объявлено во внутренней области видимости, оно *скрывает* (hidden name) такое же имя, объявленное во внешней области видимости. Имена не перегружают в областях видимости:

```
string read();
void print(const string &);
void print(double); // перегружает функцию print
void fooBar(int ival)
{
    bool read = false; // новая область видимости: скрывает
                        // предыдущее объявление имени read
    string s = read(); // ошибка: read - переменная типа bool, а не
                        // функция
    // плохой подход: обычно не следует объявлять функции в локальной
    // области видимости
    void print(int); // новая область видимости: скрывает предыдущие
                    // экземпляры функции print
    print("Value: "); // ошибка: print(const string &) скрыта
    print(ival);      // ok: print(int) видима
    print(3.14);      // ok: вызов print(int); print(double) скрыта
}
```

Большинство читателей не удивит ошибка при вызове функции `read()`. Когда компилятор обрабатывает вызов функции `read()`, он находит локальное определение имени `read`. Это имя принадлежит переменной типа `bool`, а не функции. Следовательно, вызов некорректен.

Точно тот же процесс используется при распознавании вызова функции `print()`. Объявление `print(int)` в функции `fooBar` скрывает прежнее ее объявление. В результате будет доступна только одна функция `print()`, та, которая получает один параметр типа `int`.

Когда происходит вызов функции `print()`, компилятор ищет сначала объявление этого имени. Он находит локальное объявление функции `print()`, получающей один параметр типа `int`. Как только имя найдено, компилятор игнорирует такое же имя в любой внешней области видимости. Он полагает данное объявление единственно доступным для использования. Остается лишь удостовериться в допустимости использования этого имени.



В языке C++ поиск имени осуществляется до проверки соответствия типов.

Первый вызов передает функции `print()` строковый литерал, но единственное ее объявление, находящееся в области видимости, имеет параметр типа `int`. Строковый литерал не может быть преобразован в тип `int`, поэтому вызов ошибочен. Функция `print(const string&)`, которая соответствовала бы этому вызову, скрыта и не рассматривается.

Когда происходит вызов функции `print()` с передачей аргумента типа `double()`, процесс повторяется. Компилятор находит локальное определение функции `print(int)`. Но аргумент типа `double` может быть преобразован в значение типа `int`, поэтому вызов корректен.

Если бы объявление `print(int)` находилось в той же области видимости, что и объявления других версий функции `print()`, это была бы еще одна ее перегруженная версия. В этом случае вызовы распознавались бы по-другому, поскольку компилятор видел бы все три функции:

```
void print(const string &);
void print(double); // перегружает функцию print
void print(int);    // еще один экземпляр перегрузки
void fooBar2(int ival)
{
    print("Value: "); // вызов print(const string &)
    print(ival);      // вызов print(int)
    print(3.14);      // вызов print(double)
}
```

6.5. Специальные средства

В этом разделе рассматриваются три связанных с функциями средства, которые полезны во многих, но не во всех программах: аргументы по умолчанию, встраиваемые функции и функции `constexpr`, а также некоторые другие средства, обычно используемые во время отладки.

6.5.1. Аргументы по умолчанию

Параметры некоторых функций могут обладать конкретными значениями, используемыми в большинстве, но не во всех вызовах. Такие обычно используемые значения называют *аргументом по умолчанию* (default argument). Функции с аргументами по умолчанию могут быть вызваны с ними или без них.

Например, для представления содержимого окна можно было бы использовать тип `string`. Мы могли бы хотеть, чтобы по умолчанию у окна была определенная высота, ширина и фоновый символ. Но мы могли бы также захотеть позволить пользователям использовать собственные значения, кроме значений по умолчанию. Чтобы приспособить и значение по умолчанию,

и определяемое пользователем, мы объявили бы функцию, представляющую окно, следующим образом:

```
typedef string::size_type sz; // typedef см (p 2.5.1 стр. 106)
string screen(sz ht = 24, sz wid = 80, char backgrnd = ' ');
```

Здесь мы предоставили для каждого параметра значение по умолчанию. Аргумент по умолчанию определяется как инициализатор параметра в списке параметров. Значения по умолчанию можно определить как для одного, так и для нескольких параметров. Но если у параметра есть аргумент по умолчанию, то все параметры, следующие за ним, также должны иметь аргументы по умолчанию.

Вызов функции с аргументами по умолчанию

Если необходимо использовать аргумент по умолчанию, его значение при вызове функции пропускают. Поскольку функция `screen()` предоставляет значения по умолчанию для всех параметров, мы можем вызвать ее без аргументов, с одним, двумя или тремя аргументами:

```
string window;
window = screen();           // эквивалент screen(24,80,' ')
window = screen(66);         // эквивалент screen(66,80,' ')
window = screen(66, 256);     // screen(66,256,' ')
window = screen(66, 256, '#'); // screen(66,256,'#')
```

Аргументы в вызове распознаются по позиции. Значения по умолчанию используются для аргументов, крайних справа. Например, чтобы переопределить значение по умолчанию параметра `background`, следует поставить также аргументы для параметров `height` и `width`:

```
window = screen(, , '?'); // ошибка: можно пропустить аргументы только
                           // крайние справа
window = screen('?');     // вызов screen('?',80,' ')
```

Обратите внимание, что второй вызов, передающий одно символьное значение, вполне допустим. Несмотря на допустимость, это вряд ли то, что ожидалось. Вызов допустим потому, что символ '?' имеет тип `char`, а он может быть преобразован в тип крайнего левого параметра. Это параметр типа `string::size_type`, который является целочисленным беззнаковым типом. В этом вызове аргумент типа `char` неявно преобразуется в тип `string::size_type` и передается как аргумент параметру `height`. На машине авторов символ '?' имеет шестнадцатеричное значение `0x3F`, соответствующее десятичному 63. Таким образом, этот вызов присваивает параметру `height` значение 63.

Одной из задач при разработке функции с аргументами по умолчанию является упорядочивание параметров так, чтобы те из них, для которых использование значения по умолчанию вероятней всего, располагались последними.

Объявление аргумента по умолчанию

Хотя вполне обычной практикой является объявление функции однажды в заголовке, вполне допустимо многократно объявлять ее повторно. Однако у каждого параметра может быть свое значение по умолчанию, определенное только однажды в данной области видимости. Таким образом, любое последующее объявление может добавить значение по умолчанию только для того параметра, у которого ранее не было определено значение по умолчанию. Как обычно, значения по умолчанию могут быть определены, только если у всех параметров справа уже есть значения по умолчанию. Рассмотрим следующий пример:

```
// у параметров height и width нет значений по умолчанию
string screen(sz, sz, char = ' ');
```

Нельзя изменить уже заявленное значение по умолчанию:

```
string screen(sz, sz, char = '*'); // ошибка: переобъявление
```

Но можно добавить аргумент по умолчанию следующим образом:

```
string screen(sz = 24, sz = 80, char); // ok: добавление аргументов
// по умолчанию
```

Рекомендуем

Обычно аргументы по умолчанию определяют при объявлении функции в соответствующем заголовке.

Инициализация аргумента по умолчанию

Локальные переменные не могут использоваться как аргумент по умолчанию. За исключением этого ограничения, аргумент по умолчанию может быть любым выражением, тип которого приводим к типу параметра:

```
// объявления wd, def и ht должны располагаться вне функции
sz wd = 80;
char def = ' ';
sz ht();
string screen(sz = ht(), sz = wd, char = def);
string window = screen(); // вызов screen(ht(), 80, '*')
```

Поиск имен, используемых для аргументов по умолчанию, осуществляется в пределах объявления функции. Значения, представляемые этими именами, вычисляются во время вызова:

```
void f2()
{
    def = '*'; // изменение значения аргумента по умолчанию
    sz wd = 100; // скрывает внешнее определение wd, но не изменяет
                // значение по умолчанию
    window = screen(); // вызов screen(ht(), 80, '*')
}
```


В функции `f2()` было изменено значение `def`. Вызов функции `screen` передает это измененное значение. Эта функция также объявляет локальную переменную, которая скрывает внешнюю переменную `wd`. Однако локальное имя `wd` никак не связано с аргументом по умолчанию, переданным функции `screen()`.

Упражнения раздела 6.5.1

Упражнение 6.40. Какое из следующих объявлений (если оно есть) содержит ошибку? Почему?

```
(a) int ff(int a, int b = 0, int c = 0);  
(b) char *init(int ht = 24, int wd, char bckgrnd);
```

Упражнение 6.41. Какие из следующих вызовов (если они есть) недопустимы? Почему? Какие из них допустимы (если они есть), но, вероятно, не соответствуют намерениям разработчика? Почему?

```
char *init(int ht, int wd = 80, char bckgrnd = ' ');  
(a) init(); (b) init(24,10); (c) init(14, '*');
```

Упражнение 6.42. Присвойте второму параметру функции `make_plural()` (см. раздел 6.3.2, стр. 294) аргумент по умолчанию `'s'`. Проверьте программу, выведя слова “success” и “failure” в единственном и множественном числе.

6.5.2. Встраиваемые функции и функции `constexpr`

В разделе 6.3.2 (стр. 294) приведена небольшая функция, возвращающая ссылку на более короткую строку из двух переданных ей. К преимуществам определения функции для такой маленькой операции относятся следующие.

- Обращение к функции `shorterString()` проще и понятнее, чем эквивалентное условное выражение.
- Использование функции гарантирует одинаковое поведение. Она гарантирует, что каждая проверка будет выполнена тем же способом.
- Если придется внести изменение, проще сделать это в теле функции, а не выискивать в коде программы все случаи применения эквивалентного выражения.
- Функция может быть многократно использована при написании других приложений.

Однако у функции `shorterString()` есть один потенциальный недостаток: ее вызов происходит медленнее, чем вычисление эквивалентного выражения. На большинстве машин при вызове функции осуществляется довольно много действий: перед обращением сохраняются регистры, которые необходимо будет восстановить после выхода; происходит копирование значений аргументов; управление программой переходит к новому участку кода.

Встраиваемые функции позволяют избежать дополнительных затрат на вызов

Содержимое функции, объявленной *встраиваемой* (`inline`) при компиляции, как правило, встраивается по месту вызова. Предположим, что функция `shorterString()` объявлена встраиваемой, а ее вызов имеет такой вид:

```
cout << shorterString(s1, s2) << endl;
```

При компиляции тело функции окажется встроено по месту вызова, и в результате получится нечто вроде следующего:

```
cout << (s1.size() < s2.size() ? s1 : s2) << endl;
```

Таким образом, во время выполнения удастся избежать дополнительных затрат, связанных с вызовом функции `shorterString()`.

Чтобы объявить функцию `shorterString()` встраиваемой, в определении, перед типом возвращаемого значения, располагают ключевое слово `inline`.

```
// встраиваемая версия функции сравнения двух строк
inline const string &
shorterString(const string &s1, const string &s2)
{
    return s1.size() <= s2.size() ? s1 : s2;
}
```



Объявление функции встраиваемой является только *рекомендацией* компилятору. Компилятор вполне может проигнорировать эту рекомендацию.

На самом деле механизм встраивания применяется в процессе оптимизации объектного кода, в ходе которого код небольших функций, вызов которых происходит достаточно часто, встраивается по месту вызова. Большинство компиляторов не будет встраивать рекурсивные функции. Функция на 75 строк также, вероятно, не будет встроена.

Функции `constexpr`



Функция `constexpr` — это функция, которая может быть применена в константном выражении (см. раздел 2.4.4, стр. 103). Функция `constexpr` определяется как любая другая функция, но должна соответствовать определенным ограничениям: возвращаемый тип и тип каждого параметра должны быть литералами (см. раздел 2.4.4, стр. 104), тело функции должно содержать только один оператор `return`:

```
constexpr int new_sz() { return 42; }
constexpr int foo = new_sz(); // ok: foo - константное выражение
```

Здесь функция `new_sz` определена как `constexpr`, она не получает никаких аргументов. Компилятор может проверить (во время компиляции), что вызов функции `new_sz()` возвращает константное выражение, поэтому ее можно использовать для инициализации переменной `constexpr` по имени `foo`.

Если это возможно, компилятор заменит вызов функции `constexpr` ее результирующим значением. Для этого функция `constexpr` неявно считается встраиваемой.

Тело функции `constexpr` может содержать другие операторы, если они не выполняют действий во время выполнения. Например, функция `constexpr` может содержать пустые операторы, псевдонимы типа (см. раздел 2.5.1, стр. 106) и объявления `using`.

Функции `constexpr` позволено вернуть значение, которое не является константой:

```
// scale(arg) - константное выражение, если arg - константное выражение
constexpr size_t scale(size_t cnt) { return new_sz() * cnt; }
```

Функция `scale()` возвратит константное выражение, если ее аргумент будет константным выражением, но не в противном случае:

```
int arr[scale(2)]; // ok: scale(2) - константное выражение
int i = 2;         // i - неконстантное выражение
int a2[scale(i)];  // ошибка: scale(i) - неконстантное выражение
```

Если передать константное выражение (такое как литерал 2), возвращается тоже константное выражение. В данном случае компилятор заменит вызов функции `scale()` результирующим значением.

Если происходит вызов функции `scale()` с выражением, которое не является константным (например, объект `i` типа `int`), то возвращается неконстантное выражение. Если использовать функцию `scale()` в контексте, требующем константного выражения, компилятор проверит, является ли результат константным выражением. Если это не так, то компилятор выдаст сообщение об ошибке.

Функция `constexpr` не обязана возвращать константное выражение.

Помещайте встраиваемые функции и функции `constexpr` в файлы заголовка

В отличие от других функций, встраиваемые функции и функции `constexpr` могут быть определены в программе несколько раз. В конце концов, чтобы встроить код, компилятор нуждается в определении, а не только в объявлении. Однако все определения конкретной встраиваемой функции и функции `constexpr` должны совпадать точно. В результате встраиваемые функции и функции `constexpr` обычно определяют в заголовках.

Упражнения раздела 6.5.2

Упражнение 6.43. Какое из следующих объявлений и определений имеет смысл поместить в файл заголовка, а какой — в текст файла исходного кода? Объясните почему.

- (a) `inline bool eq(const BigInt&, const BigInt&) {...}`
- (b) `void putValues(int *arr, int size);`

Упражнение 6.44. Перепишите функцию `isShorter()` из раздела 6.2.2 (стр. 279) как встраиваемую.

Упражнение 6.45. Пересмотрите функции, написанные для предыдущих упражнений, и решите, должны ли они быть определены как встраиваемые. Если да, то сделайте это. В противном случае объясните, почему они не должны быть встраиваемыми.

Упражнение 6.46. Возможно ли определить функцию `isShorter` как `constexpr`? Если да, то сделайте это. В противном случае объясните, почему нет.

6.5.3. Помощь в отладке

Для условного выполнения отладочного кода программисты C++ иногда используют подход, подобный защите заголовка (см. раздел 2.6.3, стр. 117). Идея в том, что программа будет содержать отладочный код, который выполняется только во время разработки программы. Когда приложение закончено и готово к выпуску, отладочный код исключается. Этот подход подразумевает использование двух средств препроцессора: `assert` и `NDEBUG`.

Макрос препроцессора `assert`

Макрос `assert` — это *макрос препроцессора* (preprocessor macro). Макрос препроцессора — это переменная препроцессора, действующая как встраиваемая функция. Макрос `assert` получает одно выражение и использует его как условие:

```
assert (выражение) ;
```

Если результат выражения ложь (т.е. нуль), то макрос `assert` выдает сообщение и закрывает программу. Если результат выражения — истина (т.е. он отличен от нуля), то макрос `assert` не делает ничего.

Действие макроса препроцессора подобно вызову функции. Макрос `assert` получает одно *выражение*, которое он использует как условие.

Макрос `assert` определен в заголовке `cassert`. Как уже упоминалось, относящиеся к препроцессору имена обрабатывает препроцессор, а не компилятор (см. раздел 2.3.2, стр. 89). В результате такие имена можно использовать

непосредственно, без объявления `using`. Таким образом, используется имя `assert`, а не `std::assert`, кроме того, для него не предоставляется объявление `using`.

Макрос `assert` зачастую используется для проверки “недопустимых” условий. Например, программа обработки вводимого текста могла бы проверять, что все вводимые слова длиннее некоего порогового значения. Эта программа могла бы содержать такой оператор:

```
assert(word.size() > threshold);
```

Переменная препроцессора NDEBUG

Поведение макроса `assert` зависит от состояния переменной препроцессора `NDEBUG`. Если переменная `NDEBUG` определена, макрос `assert` ничего не делает. По умолчанию переменная `NDEBUG` не определена, поэтому по умолчанию макрос `assert` выполняет проверку.

Отладку можно “выключить”, предоставив директиву `#define`, определяющую переменную `NDEBUG`. В качестве альтернативы большинство компиляторов предоставляет параметр командной строки, позволяющий определять переменные препроцессора:

```
$ CC -D NDEBUG main.C # use /D with the Microsoft compiler
```

Результат будет тот же, что и при наличии строки `#define NDEBUG` в начале файла `main.C`.

Когда переменная `NDEBUG` определена, программа во время выполнения избегает дополнительных затрат на проверку различных условий. Самих проверок во время выполнения, конечно, тоже не будет. Поэтому макрос `assert` следует использовать только для проверки того, что действительно недопустимо. Это может быть полезно при отладке программы, но не должно использоваться для замены логических проверок времени выполнения или проверки ошибок, которые должна осуществлять программа.

В дополнение к макросу `assert` можно написать собственный отладочный код, выполняющийся в зависимости от переменной `NDEBUG`. Если переменная `NDEBUG` *не определена*, код между директивами `#ifndef` и `#endif` выполняется, а в противном случае игнорируется:

```
void print(const int ia[], size_t size)
{
#ifdef NDEBUG
// __func__ - локальная статическая переменная, определенная
// компилятором. Она содержит имя функции
cerr << __func__ << ": array size is " << size << endl;
#endif
// ...
```

Здесь переменная `__func__` используется для вывода имени отлаживаемой функции. Компилятор определяет переменную `__func__` в каждой функции. Это локальный статический массив типа `const char`, содержащий имя функции.

Кроме переменной `__func__`, определяемой компилятором C++, препроцессор определяет четыре других имени, которые также могут пригодиться при отладке:

`__FILE__` строковый литерал, содержащий имя файла.

`__LINE__` целочисленный литерал, содержащий номер текущий строки.

`__TIME__` строковый литерал, содержащий файл и время компиляции.

`__DATE__` строковый литерал, содержащий файл и дату компиляции.

Эти константы можно использовать для отображения дополнительной информации в сообщениях об ошибках:

```
if (word.size() < threshold)
    cerr << "Error: " << __FILE__
        << " : in function " << __func__
        << " at line " << __LINE__ << endl
        << " Compiled on " << __DATE__
        << " at " << __TIME__ << endl
        << " Word read was \"" << word
        << "\": Length too short" << endl;
```

Если передать этой программе строку, которая короче `threshold`, то будет создано следующее сообщение об ошибке:

```
Error: wdebug.cc : in function main at line 27
    Compiled on Jul 11 2012 at 20:50:03
    Word read was "foo": Length too short
```

Упражнения раздела 6.5.3

Упражнение 6.47. Пересмотрите программу, написанную в упражнении раздела 6.3.2 (стр. 299), где использовалась рекурсия для отображения содержимого вектора так, чтобы условно отображать информацию о ее выполнении. Например, отобразите размер вектора при каждом вызове. Откомпилируйте и запустите программу с включенной отладкой и с выключенной.

Упражнение 6.48. Объясните, что делает этот цикл и стоит ли использовать в нем макрос `assert`:

```
string s;
while (cin >> s && s != sought) { } // пустое тело
assert(cin);
```



6.6. Подбор функции

Во многих (если не во всех) случаях довольно просто выяснить, какая из перегруженных версий функции будет использована при данном вызове. Но это не так просто, когда у перегруженных функций одинаковое количество параметров и когда один или несколько параметров имеют типы, связанные преобразованиями. Для примера рассмотрим следующий набор перегруженных функций и их вызов:

```
void f();  
void f(int);  
void f(int, int);  
void f(double, double = 3.14);  
f(5.6); // вызов void f(double, double)
```

Выявление кандидатов и подходящих функций

На первом этапе подбора перегруженной функции выявляют набор версий, подходящих для рассматриваемого вызова. Такие функции называются *функциями-кандидатами* (candidate function). Функция-кандидат имеет имя, указанное при вызове, и видима в точке вызова. В данном примере кандидатами являются все четыре функции по имени *f*.

На втором этапе выбора функции из набора кандидатов выявляются те, которые могут быть вызваны с аргументами данного вызова. Выбранные функции называют *подходящими* (viable function). Чтобы считаться подходящей, функция должна иметь столько же параметров, сколько аргументов передано при вызове, и тип каждого аргумента должен совпадать или допускать преобразование в тип соответствующего параметра.

При вызове `f(5.6)` две функции-кандидата можно исключить сразу из-за несоответствия количеству аргументов. Речь идет о версии без параметров и версии с двумя параметрами типа `int`. В данном случае вызов имеет только один аргумент, а эти функции не имеют их вообще или имеют два параметра соответственно.

Функция, получающая один аргумент типа `int`, и функция, получающая два аргумента типа `double`, могли бы быть подходящими. Любая из них может быть вызвана с одним аргументом. Функция, получающая два аргумента типа `double`, имеет аргумент по умолчанию, а значит, может быть вызвана с одним аргументом.



Когда у функции есть аргументы по умолчанию (см. раздел 6.5.1, стр. 308), при вызове может быть передано меньше аргументов, чем она фактически имеет.

После проверки количества аргументов, позволяющей выявить функции, подходящие потенциально, проверяется соответствие типов параметров

функций типам аргументов, переданных при вызове. Как и при любом обращении, тип аргумента может либо совпадать, либо допускать преобразование в тип параметра. В данном случае подходят обе оставшиеся функции.

- Функция `f(int)` является подходящей потому, что аргумент типа `double` может быть неявно преобразован в параметр типа `int`.
- Функция `f(double, double)` также является подходящей потому, что для второго параметра задано значение по умолчанию, а первый параметр имеет тип `double`, который точно соответствует типу аргумента.



Если никаких подходящих функций не обнаружено, компилятор выдает сообщение об ошибке.

Поиск наилучшего соответствия, если он есть

На третьем этапе подбора перегруженной функции выясняется, какая из допустимых функций наилучшим образом соответствует вызову. Этот процесс анализирует каждый аргумент вызова и выбирает подходящую функцию (или функции), для которой соответствие параметра аргументу является наилучшим. Подробно критерии наилучшего соответствия рассматриваются в следующем разделе, а пока достаточно знать, что чем ближе типы аргумента и параметра друг к другу, тем лучше соответствие.

В данном случае существует только один (явный) аргумент, который имеет тип `double`. При вызове версии `f(int)` аргумент преобразуется из типа `double` в тип `int`. Вторая подходящая функция, `f(double, double)`, точно соответствует типу этого аргумента. Поскольку точное соответствие лучше соответствия требующего преобразования, компилятор предпочитает версию с двумя параметрами типа `double`. Для второго, недостающего аргумента компилятор добавит аргумент по умолчанию.

Подбор перегруженной версии с несколькими параметрами

Если у функции два или несколько аргументов, подбор подходящей версии усложняется. Предположим, что функции имеют то же имя `f`, но анализируется следующий вызов:

```
f(42, 2.56);
```

Набор подходящих функций выявляется, как прежде. Компилятор выбирает те версии функции, которые имеют необходимое количество параметров, типы которых соответствуют типам аргументов. В данном случае в набор подходящих вошли функции `f(int, int)` и `f(double, double)`. Затем компилятор перебирает аргументы один за одним и определяет, какая из версий функций имеет наилучшее соответствие. Наилучше соответствующая функция та, для которой единственной выполняются следующие условия.

- Соответствие по каждому аргументу не хуже, чем у остальных подходящих функций.
- По крайней мере у одного аргумента соответствие лучше, чем у остальных подходящих функций.

Если после просмотра всех аргументов не было найдено ни одной функции, которая считалась бы наилучше соответствующей, компилятор сообщает об ошибке неоднозначности вызова.

В рассматриваемом примере вызова анализ лишь первого аргумента для версии `f(int, int)` функции `f()` обнаруживает точное соответствие. При анализе второй версии функции `f()` оказывается, что аргумент `42` типа `int` следует преобразовать в значение типа `double`. Соответствие в результате встроенного преобразования хуже, чем точное. Таким образом, рассматривая только этот параметр, лучше соответствует та версия функции `f()`, которая обладает двумя параметрами типа `int`, а не двумя параметрами типа `double`.

Но при переходе ко второму аргументу оказывается, что версия функции `f()` с двумя параметрами типа `double` точно соответствует аргументу `2.56`. Вызов версии функции `f()` с двумя параметрами типа `int` потребует преобразования аргумента `2.56` из типа `double` в тип `int`. Таким образом, при рассмотрении только второго параметра версия `f(double, double)` функции `f()` имеет лучшее соответствие.

Компилятор отклонит этот вызов, поскольку он неоднозначен: каждая подходящая функция является лучшим соответствием по одному из аргументов. Было бы заманчиво обеспечить соответствие за счет явного приведения типов (см. раздел 4.11.3, стр. 223) одного из аргументов. Но в хорошо спроектированных системах в приведении аргументов не должно быть необходимости.



При вызове перегруженных функций приведения аргументов практически не нужны: потребность в приведении означает, что наборы параметров перегруженных функций проработаны плохо.

Упражнения раздела 6.6

Упражнение 6.49. Что такое функция-кандидат? Что такое подходящая функция?

Упражнение 6.50. С учетом приведенных на стр. 317 объявлений функции `f()` перечислите подходящие функции для каждого из следующих вызовов. Укажите наилучше соответствие, или если его нет, то из-за отсутствия соответствия или неоднозначности вызова?

- (a) `f(2.56, 42)` (b) `f(42)` (c) `f(42, 0)` (d) `f(2.56, 3.14)`

Упражнение 6.51. Напишите все четыре версии функции `f()`. Каждая из них должна выводить собственное сообщение. Проверьте свои ответы на предыдущее упражнение. Если ответы были неправильными, перечитайте этот раздел и выясните, почему вы ошиблись.



6.6.1. Преобразование типов аргументов

Чтобы определить наилучшее соответствие, компилятор ранжирует преобразования, применяемые для приведения типа аргумента к типу соответствующего ему параметра. Преобразования ранжируются в порядке убывания следующим образом.

1. Точное соответствие. Типы аргумента и параметра совпадают в случае, если:
 - типы аргумента и параметра идентичны;
 - аргумент преобразуется из типа массива или функции в соответствующий тип указателя. (Указатели на функции рассматриваются в разделе 6.7, стр. 322);
 - аргумент отличается наличием или отсутствием спецификатора `const` верхнего уровня.
2. Соответствие в результате преобразования констант (см. раздел 4.11.2, стр. 222).
3. Соответствие в результате преобразования (см. раздел 4.11.1, стр. 219).
4. Соответствие в результате арифметического преобразования (см. раздел 4.11.1, стр. 219) или преобразования указателя (см. раздел 4.11.2, стр. 221).
5. Соответствие в результате преобразования класса (раздел 14.9, стр. 732).



Соответствие, требующее приведения и (или) целочисленного преобразования



В контексте соответствия функций приведение и преобразование встроенных типов может привести к удивительным результатам. К счастью, в хорошо разработанных системах редко используют функции с параметрами, столь похожими, как в следующих примерах.

При анализе вызова следует помнить, что малые целочисленные типы всегда преобразуются в тип `int` или больший целочисленный тип. Рассмотрим две функции, одна из которых получает тип `int`, а вторая тип `short`, версия `short` будет вызвана только со значениями типа `short`. Даже при том, что

меньшие целочисленные значения могли бы быть ближе к соответствию, эти значения преобразуются в тип `int`, тогда как вызов версии `short` потребовал бы преобразования:

```
void ff(int);
void ff(short);
ff('a'); // тип char приводится к int, поэтому применяется f(int)
```

Все целочисленные преобразования считаются эквивалентными друг другу. Преобразование из типа `int` в `unsigned int`, например, не имеет преимуществ перед преобразованием типа `int` в `double`. Рассмотрим конкретный пример.

```
void manip(long);
void manip(float);
manip(3.14); // ошибка: неоднозначный вызов
```

Литерал `3.14` имеет тип `double`. Этот тип может быть преобразован или в тип `long`, или в тип `float`. Поскольку возможны два целочисленных преобразования, вызов неоднозначен.

Соответствие функций и константные аргументы

Когда происходит вызов перегруженной функции, различие между версиями которой заключается в том, указывает ли параметр (или ссылается) на константу, компилятор способен различать, является ли аргумент константным или нет:

```
Record lookup(Account&);           // функция, получающая ссылку на Account
Record lookup(const Account&);      // новая функция, получающая ссылку на
                                   // константу

const Account a;
Account b;
lookup(a); // вызов lookup(const Account&)
lookup(b); // вызов lookup(Account&)
```

В первом вызове передается константный объект `a`. Нельзя связать простую ссылку с константным объектом. В данном случае единственная подходящая функция — версия, получающая ссылку на константу. Кроме того, этот вызов точно соответствует аргументу `a`.

Во втором вызове передается неконстантный объект `b`. Для этого вызова подходят обе функции. Аргумент `b` можно использовать для инициализации ссылки константного или неконстантного типа. Но инициализация ссылки на константу неконстантным объектом требует преобразования. Версия, получающая неконстантный параметр, является точным соответствием для объекта `b`. Следовательно, неконстантная версия предпочтительней.

Параметры в виде указателя работают подобным образом. Если две функции отличаются только тем, указывает ли параметр на константу или не кон-

станту, компилятор на основании константности аргумента вполне может решить, какую версию функции использовать: если аргумент является указателем на константу, то вызов будет соответствовать версии, получающей тип `const*`; в противном случае, если аргумент — указатель на не константу, вызывается версия, получающая простой указатель.

Упражнения раздела 6.6.1

Упражнение 6.52. Предположим, что существуют следующие объявления:

```
void manip(int, int);
double dobj;
```

Каков порядок (см. раздел 6.6.1, стр. 320) преобразований в каждом из следующих обращений?

(a) `manip('a', 'z');` (b) `manip(55.4, dobj);`

Упражнение 6.53. Объясните назначение второго объявления в каждом из следующих наборов. Укажите, какие из них (если они есть) недопустимы.

```
(a) int calc(int&, int&);
    int calc(const int&, const int&);
(b) int calc(char*, char*);
    int calc(const char*, const char*);
(c) int calc(char*, char*);
    int calc(char* const, char* const);
```

6.7. Указатели на функции

Указатель на функцию (function pointer) содержит адрес функции, а не объекта. Подобно любому другому указателю, указатель на функцию имеет вполне определенный тип. Тип функции определен типом ее возвращаемого значения и списком параметров. Имя функции не является частью ее типа.

```
// сравнивает длины двух строк
bool lengthCompare(const string &, const string &);
```

Эта функция имеет тип `bool(const string&, const string&)`. Чтобы объявить указатель, способный указывать на эту функцию, достаточно расположить указатель вместо имени функции:

```
// pf указывает на функцию, получающую две константные ссылки
// на строки и возвращающую значение типа bool
bool (*pf)(const string &, const string &); // не инициализирован
```

Просматривая объявление с начала, можно заметить, что имени `pf` предшествует знак `*`, следовательно, `pf` — указатель. Справа расположен список параметров, означая, что `pf` указывает на функцию. Глядя влево, можно заметить, что возвращаемым типом функции является `bool`. Таким образом, ука-

затель `pf` указывает на функцию, которая имеет два параметра типа `const string&` и возвращает значение типа `bool`.



Круглые скобки вокруг части `*pf` необходимы. Без них получится объявление функции `pf()`, возвращающей указатель на тип `bool`:

```
// объявление функции pf(), возвращающей указатель на тип bool
bool *pf(const string &, const string &);
```

Использование указателей на функцию

При использовании имени функции как значения функция автоматически преобразуется в указатель. Например, адрес функции `lengthCompare()` можно присвоить указателю `pf` следующим образом:

```
pf = lengthCompare; // pf теперь указывает на функцию lengthCompare
pf = &lengthCompare; // эквивалентное присвоение: оператор обращения к
// адресу необязателен
```

Кроме того, указатель на функцию можно использовать для вызова функции, на которую он указывает. Это можно сделать непосредственно, обращение к значению указателя там не обязательно:

```
bool b1 = pf("hello", "goodbye"); // вызов lengthCompare
bool b2 = (*pf)("hello", "goodbye"); // эквивалентный вызов
bool b3 = lengthCompare("hello", "goodbye"); // эквивалентный вызов
```

Преобразование указателя на один тип функции в указатель на другой тип функции невозможно. Однако для обозначения того, что указатель не указывает на функцию, ему можно присвоить `nullptr` (см. раздел 2.3.2, стр. 89) или целочисленное константное выражение, означающее нуль:

```
string::size_type sumLength(const string&, const string&);
bool cstringCompare(const char*, const char*);
pf = 0; // ok: pf не указывает на функцию
pf = sumLength; // ошибка: разные типы возвращаемого значения
pf = cstringCompare; // ошибка: разные типы параметров
pf = lengthCompare; // ok: типы функции и указателя совпадают точно
```

Указатели на перегруженные функции

Как обычно, при использовании перегруженной функции применяемую версию должен прояснить контекст, в котором она используется. Вот объявление указателя на перегруженную функцию:

```
void ff(int*);
void ff(unsigned int);
void (*pff)(unsigned int) = ff; // pff указывает на ff(unsigned)
```

Компилятор использует тип указателя для выявления используемой версии перегруженной функции. Тип указателя должен точно соответствовать одной из версий перегруженной функции:

```
void (*pf2)(int) = ff;    // ошибка: нет версии с точно таким списком
                        // параметров
double (*pf3)(int*) = ff; // ошибка: тип возвращаемого значения
                        // функций ff и pf3 не совпадают
```

Указатель на функцию как параметр

Подобно массивам (см. раздел 6.2.4, стр. 283), нельзя определить параметры типа функции, но можно создать параметр, являющийся указателем на функцию. Как и в случае с массивами, можно создать параметр, который выглядит как тип функции, но обрабатывается как указатель:

```
// третий параметр имеет тип функции и автоматически обрабатывается как
// указатель на функцию
void useBigger(const string &s1, const string &s2,
               bool pf(const string &, const string &));
// эквивалентное объявление: параметр явно определен как указатель
// на функцию
void useBigger(const string &s1, const string &s2,
               bool (*pf)(const string &, const string &));
```

При передаче функции как аргумента это можно сделать непосредственно. Аргумент будет автоматически преобразован в указатель:

```
// автоматическое преобразование функции lengthCompare в указатель
// на нее
useBigger(s1, s2, lengthCompare);
```

Как можно заметить в объявлении функции `useBigger()`, написание указателей на тип функций быстро становится утомительным. Псевдонимы типа (см. раздел 2.5.1, стр. 106), а также спецификатор `decltype` (см. раздел 2.5.3, стр. 110) позволяют упростить код, который использует указатели на функции:

```
// Func и Func2 имеют тип функции
typedef bool Func(const string&, const string&);
typedef decltype(lengthCompare) Func2;           // эквивалентный тип
// FuncP и FuncP2 имеют тип указателя на функцию
typedef bool(*FuncP)(const string&, const string&);
typedef decltype(lengthCompare) *FuncP2;         // эквивалентный тип
```

Здесь при определении типов использовано ключевое слово `typedef`. И `Func`, и `Func2` являются типами функций, тогда как `FuncP` и `FuncP2` — типы указателя. Следует заметить, что спецификатор `decltype` возвращает тип функции; автоматического преобразования в указатель не происходит. Поскольку спецификатор `decltype` возвращает тип функции, при необходимости получить указатель следует добавить символ `*`. Можно повторно объявить функцию `useBigger()`, используя любой из этих типов:

```
// эквивалентные объявления useBigger с использованием псевдонимов типа
void useBigger(const string&, const string&, Func);
void useBigger(const string&, const string&, FuncP2);
```

Оба объявления объявляют ту же функцию. В первом случае компилятор автоматически преобразует тип функции, представленный именем `Func`, в указатель.

Возвращение указателя на функцию

Подобно массивам (см. раздел 6.3.3, стр. 299), нельзя вернуть тип функции, но можно вернуть указатель на тип функции. Точно так же тип возвращаемого значения следует писать как тип указателя; компилятор не будет автоматически рассматривать тип возвращаемого значения функции как соответствующий тип указателя. Как и при возвращении массива, безусловно, проще всего объявить функцию, которая возвращает указатель на функцию, при помощи псевдонима типа:

```
using F = int(int*, int);      // F - тип функции, а не указатель
using PF = int(*) (int*, int); // PF - тип указателя
```

Здесь для определения `F` как типа функции и `PF` как указателя на тип функции было использовано объявление псевдонима типа (см. раздел 2.5.1, стр. 106). Имейте в виду, что в отличие от параметров, имеющих тип функции, тип возвращаемого значения не преобразуется автоматически в тип указателя. Следует явно определить, что тип возвращаемого значения является типом указателя:

```
PF f1(int); // ok: PF - указатель на функцию; f1 возвращает указатель
              // на функцию
F f1(int);  // ошибка: F - тип функции; f1 не может вернуть функцию
F *f1(int); // ok: явное определение типа возвращаемого значения как
              // указателя на функцию
```

Конечно, функцию `f1()` также можно объявить непосредственно:

```
int (*f1(int))(int*, int);
```

Читая это объявление изнутри наружу, можно заметить у `f1` список параметров, таким образом, `f1` — это функция. Имени `f1` предшествует знак `*`, следовательно, функция `f1()` возвращает указатель. У типа самого указателя тоже есть список параметров, таким образом, указатель указывает на функцию. Эта функция возвращает тип `int`.

И наконец, следует обратить внимание на то, что объявления функций, которые возвращают указатель на функцию, можно упростить при помощи замыкающего типа возвращаемого значения (см. раздел 6.3.3, стр. 301):

```
auto f1(int) -> int (*) (int*, int);
```

Использование спецификаторов `auto` и `decltype` для типов указателей на функции

Если известно, какую функцию (функции) следует возвратить, можно использовать спецификатор `decltype` для упрощения записи типа возвращаемого значения в виде указателя на функцию. Предположим, например, что имеются две функции, обе возвращают тип `string::size_type` и имеют два параметра типа `const string&`. Можно написать третью функцию, которая получает параметр типа `string` и возвращает указатель на одну из следующих двух функций следующим образом:

```
string::size_type sumLength(const string&, const string&);
string::size_type largerLength(const string&, const string&);
// в зависимости от значения строкового параметра функция getFcn
// возвращает указатель на sumLength или largerLength
decltype(sumLength) *getFcn(const string &);
```

Единственная сложность в объявлении функции `getFcn()` — это необходимость помнить, что при применении спецификатора `decltype` к функции она возвращает тип функции, а не указатель на тип функции. Чтобы получить указатель, а не функцию, следует добавить знак `*`.

Упражнения раздела 6.7

Упражнение 6.54. Напишите объявление функции, получающей два параметра типа `int`, и возвращающей тип `int`. Объявите также вектор, элементами которого является тип указателя на эту функцию.

Упражнение 6.55. Напишите четыре функции, которые добавляют, вычитают, умножают и делят два значения типа `int`. Сохраните указатели на эти значения в векторе из предыдущего упражнения.

Упражнение 6.56. Обратитесь к каждому элементу вектора и выведите результат.

Резюме

Функции представляют собой именованные блоки действий, применяемые для структурирования даже небольших программ. При их определении указывают тип возвращаемого значения, имя, список параметров (возможно, пустой) и тело функции. Тело функции — это блок операторов, выполняемых при вызове функции. Переданные функции при вызове аргумента должны быть совместимы с типами соответствующих параметров.

В языке C++ функции могут быть перегружены. То есть одинаковое имя может быть использовано при определении разных функций, отличающихся

количеством или типами параметров. На основании переданных при вызове аргументов компилятор автоматически выбирает наиболее подходящую версию функции. Процесс выбора правильной версии из набора перегруженных функций называют подбором функции с наилучшим соответствием.

Термины

Автоматический объект (automatic object). Объект, являющийся для функции локальным. Автоматические объекты создаются и инициализируются при каждом обращении и удаляются по завершении блока, в котором они были определены.

Аргумент (argument). Значение, предоставляемое при вызове функции для инициализации соответствующих параметров.

Аргумент по умолчанию (default argument). Значение, определенное для использования, когда аргумент пропущен при вызове функции.

Бесконечная рекурсия (recursion loop). Когда у рекурсивной функции отсутствует условие остановки, она вызывает сама себя до исчерпания стека программы.

Встраиваемая функция (inline function). Функция, тело которой встраивается по месту обращения, если это возможно. Встраиваемые функции позволяют избежать обычных дополнительных затрат, поскольку их вызов заменяет код тела функции.

Вызов по значению (call by value). См. передача по значению.

Вызов по ссылке (call by reference). См. передача по ссылке.

Исполняемый файл (executable file). Файл, содержащий программный код, который может быть выполнен операционной системой.

Класс `initializer_list`. Библиотечный класс, представляющий разделяемый запятыми список объектов одинакового типа, заключенный в фигурные скобки.

Компоновка (link). Этап компиляции, на котором несколько объектных файлов объединяются в исполняемую программу.

Локальная переменная (local variable). Переменные, определенные в блоке.

Локальный статический объект (local static object). Локальный объект, который создается и инициализируется только один раз перед первым вызовом функции, в которой используется ее значение. Значение локального статического объекта сохраняется на протяжении всех вызовов функции.

Макрос `assert`. Макрос препроцессора, который получает одно выражение, используемое в качестве условия. Если переменная препроцессора `NDEBUG` не определена, макрос `assert` проверяет условие. Если оно ложно, макрос `assert` выводит сообщение и завершает программу.

Макрос препроцессора (preprocessor macro). Средство препроцессора, ведущее себя как встраиваемая функция. Кроме макроса `assert`, современные программы C++ очень редко используют макросы препроцессора.

Наилучшее соответствие (best match). Функция, выбранная для вызова из набора перегруженных версий. Если наилучшее соответствие существует, выбранная функция лучше остальных подходит по крайней мере для одного аргумента вызова и не хуже остальных версий для оставшейся части аргументов.

- Неоднозначный вызов** (ambiguous call). Ошибка времени компиляции, происходящая при поиске подходящей функции, когда две или более функции обеспечивают одинаково хорошее соответствие для вызова.
- Объектный код** (object code). Формат, в который компилятор преобразует исходный код.
- Объектный файл** (object file). Файл, содержащий объектный код, созданный компилятором из предоставленного файла исходного кода. Исполняемый файл создается в результате компоновки одного или нескольких объектных файлов.
- Оператор** (). Оператор вызова. Запускает функцию на выполнение. Круглые скобки, следующие за именем функции или указателем на функцию, заключают разделяемый запятыми список аргументов, который может быть пуст.
- Отсутствие соответствия** (no match). Ошибка времени компиляции, происходящая при поиске подходящей функции, когда не обнаружено ни одной функции с параметрами, которые соответствуют аргументам при данном вызове.
- Параметр** (parameter). Локальная переменная, объявляемая в списке параметров функции. Параметры инициализируются аргументами, предоставляемыми при каждом вызове функции.
- Перегруженная функция** (overloaded function). Функция, которая имеет то же имя, что и по крайней мере одна другая функция. Перегруженные функции должны отличаться по количеству или типу их параметров.
- Передача по значению** (pass by value). Способ передачи аргументов параметрам не ссылочного типа. Не ссылочный параметр — это копия значения соответствующего аргумента.
- Передача по ссылке** (pass by reference). Способ передачи аргументов параметрам ссылочного типа. Ссылочные параметры работают так же, как и любая другая ссылка; параметр связан со своим аргументом.
- Замыкающий тип возвращаемого значения** (trailing return type). Тип возвращаемого значения, определенный после списка параметров.
- Подбор функции** (function matching). Процесс, в ходе которого компилятор ассоциирует вызов функции с определенной версией из набора перегруженных функций. При подборе функции используемые в обращении аргументы сравниваются со списком параметров каждой версии перегруженной функции.
- Подходящая функция** (viable function). Подмножество перегруженных функций, которые могли бы соответствовать данному вызову. У подходящих функций количество параметров совпадает с количеством переданных при обращении аргументов, а тип каждого аргумента может быть преобразован в тип соответствующего параметра.
- Поиск перегруженной функции** (overload resolution). См. подбор функции.
- Продолжительность существования объекта** (object lifetime). Каждый объект характеризуется своей продолжительностью существования. Нестатические объекты, определенные в блоке, существуют от момента их определения и до конца блока, в котором они определены. Глобальные объекты создаются во время запуска программы. Локальные статические объекты создаются прежде, чем выполнение впервые пройдет через определение объекта. Глобальные объекты и локальные статические объекты удаляются по завершении функции `main()`.
- Прототип функции** (function prototype). Синоним объявления функции. В прототипе указано имя, тип возвращаемого значения и типы параметров функции. Чтобы

функцию можно было вызвать, ее прототип должен быть объявлен перед точкой обращения.

Раздельная компиляция (separate compilation). Способность разделить программу на несколько отдельных файлов исходного кода.

Рекурсивная функция (recursive function). Функция, которая способна вызвать себя непосредственно или косвенно.

Скрытое имя (hidden name). Имя, объявленное в области видимости, но скрытое ранее объявленной сущностью с тем же именем, объявленным вне этой области видимости.

Тело функции (function body). Блок операторов, в котором определены действия функции.

Тип возвращаемого значения (return type). Часть объявления функции, определяющее тип значения, которое возвращает функция.

Функция constexpr. Функция, способная вернуть константное выражение. Функция constexpr неявно является встраиваемой.

Функция (function). Именованный блок действий.

Функция-кандидат (candidate function). Одна из функций набора, рассматриваемая при поиске соответствия вызову функции. Кандидатами считаются все функции, объявленные в области видимости обращения, имя которых совпадает с используемым в обращении.

